



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

Evolución y Optimización de
la Digitalización del Boletín
de Turismo de la Provincia de
Burgos
Documentación Técnica



Presentado por Nicolás Pérez Ibáñez
en Universidad de Burgos — 7 de julio de 2026

Tutor: Bruno Baruque Zanón

Cotutor: Julio César Puche Regaliza

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	8
Apéndice B Especificación de Requisitos	15
B.1. Introducción	15
B.2. Objetivos generales	15
B.3. Catálogo de requisitos	16
B.4. Especificación de requisitos	20
Apéndice C Especificación de diseño	53
C.1. Introducción	53
C.2. Diseño de datos	53
C.3. Diseño arquitectónico	56
C.4. Diseño procedimental	58
Apéndice D Documentación técnica de programación	63
D.1. Introducción	63
D.2. Estructura de directorios y archivos	63
D.3. Manual del programador	66

D.4. Compilación, instalación y ejecución del proyecto	85
D.5. Pruebas del sistema	95
Apéndice E Documentación de usuario	103
E.1. Introducción	103
E.2. Requisitos de usuarios	103
E.3. Instalación	104
E.4. Manual de usuario	104
Apéndice F Anexo de sostenibilización curricular	117
F.1. Introducción	117
F.2. Conclusión y reflexión personal	119
Bibliografía	121

Índice de figuras

B.1. Diagrama completo de casos de uso del pipeline ETL.	20
C.1. Diagrama Entidad-Relación del modelo de datos en BigQuery. .	56
C.2. Diagrama de arquitectura del proyecto.	58
C.3. Diagrama de secuencia: Inicialización y descubrimiento de puntos de interés (Fase A).	59
C.4. Diagrama de secuencia: Extracción de reseñas, procesamiento NLP y persistencia (Fase B).	60
C.5. Diagrama de secuencia completo.	61
E.1. Pantalla de vista general (Dashboard Overview).	106
E.2. Pantalla de análisis de Municipios y Categorías.	107
E.3. Primera pantalla del análisis del índice TORI.	108
E.4. Segunda pantalla del análisis del índice TORI.	109
E.5. Pantalla de análisis geográfico y espacial.	111
E.6. Pantalla de datos generales y características de los POIs.	112
E.7. Pantalla de datos generales y características de los POIs.	113
E.8. Pantalla de análisis temporal e histórico de reseñas.	114
E.9. Pantalla de datos demográficos de las reseñas.	115
E.10. Pantalla de análisis de sentimiento de las opiniones.	116

Índice de tablas

A.1. Planes de precios de Apify (Formato: USD / EUR)	10
B.1. CU-1 Configurar el sistema.	21
B.2. CU-1.1 Configurar márgenes de actualización.	22
B.3. CU-1.2 Configurar filtros de ejecución.	23
B.4. CU-1.3 Configurar criterios de ordenación.	24
B.5. CU-1.4 Configurar límites demográficos.	25
B.6. CU-1.5 Configurar el nivel de detalle de los logs.	26
B.7. CU-2 Controlar la ejecución secuencial del pipeline.	27
B.8. CU-2.1 Ejecutar de forma programada.	28
B.9. CU-2.2 Ejecutar de forma manual.	29
B.10. CU-3 Extraer información.	30
B.11. CU-3.1 Recorrer municipios en bucle.	31
B.12. CU-3.2 Variar la extracción según la clasificación del municipio.	32
B.13. CU-3.3 Recorrer categorías en bucle.	33
B.14. CU-3.4 Extraer puntos de interés.	34
B.15. CU-3.5 Filtrar por código postal.	35
B.16. CU-3.6 Extraer reseñas.	36
B.17. CU-4 Procesar y limpiar reseñas.	37
B.18. CU-4.1 Eliminar reseñas sin valor.	38
B.19. CU-4.2 Traducir reseñas.	39
B.20. CU-4.3 Detectar el género.	40
B.21. CU-4.4 Analizar el sentimiento.	41
B.22. CU-5 Almacenar y optimizar.	42
B.23. CU-5.1 Evitar descargas redundantes.	43
B.24. CU-5.2 Calcular el TORI.	44
B.25. CU-5.3 Gestionar categorías nuevas.	45
B.26. CU-5.4 Cargar datos en BigQuery.	46

B.27.CU-5.5 Limpiar tablas temporales.	47
B.28.CU-5.6 Registrar marcas de control.	48
B.29.CU-6 Monitorizar la ejecución.	49
B.30.CU-6.1 Mostrar logs de ejecución.	50
B.31.CU-6.2 Almacenar historial de logs.	51
B.32.CU-7 Visualizar datos.	52
D.1. Matriz de correspondencia entre pruebas y requisitos funcionales.	97
D.2. Tabla de prueba PE-01.	98
D.3. Tabla de prueba PE-02.	99
D.4. Tabla de prueba PE-03.	100
D.5. Tabla de prueba PE-04.	101

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este anexo detalla el marco organizativo y operativo del proyecto. En primer lugar, se expone la planificación temporal realizada durante el periodo de desarrollo del TFG. En segundo lugar, se presenta el estudio de viabilidad del sistema, analizando la sostenibilidad financiera de las infraestructuras cloud elegidas y el cumplimiento normativo en materia de captura automatizada de datos y protección de la privacidad. Con ello, se justifica formalmente la viabilidad y la hoja de ruta seguida durante el proyecto.

A.2. Planificación temporal

Para el desarrollo de este proyecto, se ha seguido una metodología ágil basada en *Sprints* de trabajo de aproximadamente 2 semanas, gestionando las tareas a través de Issues y un tablero Kanban mediante la herramienta ZenHub integrada en GitHub. Esto ha permitido adaptar el desarrollo del pipeline de forma iterativa según surgían imprevistos o nuevas ideas para mejorar los resultados.

A continuación, se detallan las fases y los bloques de trabajo principales en los que se organizó el periodo de desarrollo:

Sprint 1: Configuración inicial del entorno (19 de febrero - 3 de marzo)

El primer sprint se centró en el establecimiento del entorno de desarrollo base para el proyecto. Las tareas incluyeron la inicialización del repositorio de código, la gestión de las dependencias necesarias y la obtención de las credenciales necesarias para interactuar con las APIs externas. Asimismo, se realizó un análisis detallado de la documentación técnica del proyecto anterior con el objetivo de identificar puntos de mejora y prever limitaciones técnicas en las fases de extracción de datos.

- Lectura y análisis de la documentación técnica.
- Configuración inicial del entorno y repositorio.
- Instalación de dependencias.
- Obtener claves de API (Google y Apify).

Sprint 2: Prototipado de almacenamiento y migración de datos (3 de marzo - 17 de marzo)

Este sprint se enfocó en la evaluación de los sistemas de gestión de bases de datos y la portabilidad de la información. Para ello, se desplegó e instaló un servidor local en MySQL con el objetivo de testear la adaptación de los scripts de Python preexistentes. Asimismo, se realizó una migración de prueba desde la infraestructura previa en Azure (SQL Server) hacia el entorno local para validar la integridad de los datos. Esta fase fue crucial para contrastar la viabilidad del almacenamiento relacional local frente a las futuras soluciones definitivas en la nube, además de asentar las bases de la documentación en Overleaf y la planificación general.

- Instalar y configurar servidor local MySQL.
- Adaptar scripts de Python para integración con MySQL.
- Iniciar proyecto en Overleaf.
- Crear planificación general de Sprints.
- Migración de Base de Datos de Azure (SQL Server) a MySQL Local.

Sprint 3: Investigación de herramientas de scraping y planificación (18 de marzo - 24 de marzo)

El tercer sprint se dedicó al estudio exhaustivo y evaluación técnica de las alternativas tecnológicas para la extracción automatizada, el procesamiento de datos y la infraestructura del sistema. Se analizaron herramientas y plataformas como Apify, Google Cloud Platform (GCP) y GitHub Actions con el objetivo de proponer una arquitectura en la nube robusta que superase las limitaciones de los sistemas locales. En paralelo, se configuró el ecosistema de gestión ágil en ZenHub mediante la configuración de los sistemas de estimación de esfuerzo, la personalización del etiquetado de tareas y la vinculación de recursos en el repositorio principal.

- Vincular enlace de Overleaf en el README.
- Configuración del entorno de trabajo y sistema de estimación de esfuerzo en Zenhub.
- Estudio de alternativas para la extracción, automatización e infraestructura de datos.
- Creación de etiquetas necesarias para los issues planificados en los sprints.

Sprint 4: Reestructuración y prueba de flujo completo (25 de marzo - 9 de abril)

El objetivo principal de este sprint fue consolidar la nueva arquitectura del proyecto y verificar su funcionamiento integral. Durante esta fase, se realizó una limpieza profunda del repositorio, el diseño del diagrama de arquitectura y la actualización de la documentación técnica en el archivo README. Asimismo, se llevó a cabo una prueba funcional mediante un mini-scrapers para confirmar que el flujo completo procesaba los datos correctamente, abarcando desde la extracción mediante la API hasta la visualización final en Looker Studio, pasando por Docker, GitHub Actions y Google BigQuery. El objetivo principal fue asegurar que la infraestructura y las herramientas estuvieran integradas antes de comenzar con la extracción masiva de datos.

- Configuración de infraestructura en Google Cloud y esquema BigQuery.
- Configuración de seguridad y secretos.

- Desarrollo de Scraper.
- Implementación de Docker para entornos de scraping.
- Automatización CI/CD: GitHub Actions Workflow.
- Prueba Comparativa: Google Cloud Functions.
- Dashboard de prueba en Looker Studio.
- Reestructuración y limpieza del repositorio.
- Replanificación de los sprints del proyecto.

Sprint 5: Implementación del scraper y validación del modelo de análisis de sentimiento (10 de abril - 21 de abril)

Con la arquitectura ya validada, el quinto sprint se dedicó al desarrollo de la base del scraper definitivo, ampliando el rango de búsqueda a los municipios de la provincia de Burgos e incluyendo la lógica de traducción y limpieza de las reseñas obtenidas. Durante esta fase, se realizó un rediseño en la arquitectura de datos para separar la gestión de los puntos de interés (POIs) de la tabla de reseñas. Además, se llevó a cabo la integración y verificación de la librería de procesamiento del lenguaje natural con los datos reales extraídos, asegurando una correcta clasificación del análisis de sentimiento. El objetivo principal fue consolidar el núcleo del proyecto para garantizar la obtención de datos limpios, enriquecidos y analizados de forma automatizada.

- Rediseño de arquitectura de datos: Separación POIs y Reseñas.
- Implementación del scraper ampliado a la provincia de Burgos.
- Aplicar limpieza y traducción de reseñas en el scraper.
- Integración del análisis de sentimiento de las reseñas en el scraper.

Sprint 6: Fase de experimentación, segmentación y optimización de costes del scraper (22 de abril - 5 de mayo)

El sexto sprint se centró en la evaluación del rendimiento y la viabilidad económica del pipeline de extracción de datos. Durante este periodo, se

realizaron pruebas iterativas para obtener estimaciones globales de los costes operativos del scraper. Asimismo, se implementó una estrategia de optimización basada en la segmentación de los municipios de la provincia según su volumen de población. Adicionalmente, se avanzó en la optimización del barrido geográfico en entornos urbanos de alta densidad para asegurar una captura de datos equilibrada. El objetivo principal de esta fase fue acotar el comportamiento del scraper y asentar las bases del control de costes y la eficiencia del proyecto antes de proceder al cierre del código.

- Refactorización del scraper al patrón pipeline.
- Ajuste de categorías de búsqueda y prueba de eficiencia.
- Optimización de costes mediante segmentación por población.
- Hacer pruebas para obtener estimaciones globales de costes aproximadas.
- Optimización del barrido geográfico en entornos urbanos.

Sprint 7: Continuación del bloque de extracción: refactorización estructural, robustez e infraestructura (6 de mayo - 19 de mayo)

La continuidad del desarrollo del bloque de extracción y el procesamiento de datos centraron los esfuerzos de este periodo, enfocándose en dotar al sistema de una mayor estabilidad y robustez técnica. Para asegurar la tolerancia a fallos, se implementó un tratamiento de errores exhaustivo dentro del scraper y se actualizó la infraestructura base del proyecto, adaptando los contenedores Docker, las dependencias del sistema y los flujos automatizados de GitHub Actions. En paralelo, se llevó a cabo la mejora estructural de la base de datos en Google BigQuery mediante la refactorización del esquema de la tabla de puntos de interés para integrarla en un modelo relacional por IDs de categoría. Finalmente, se aislaron los parámetros operativos mediante una arquitectura de configuración externa y se desplegó un sistema de trazabilidad mediante logs, dejando el pipeline optimizado para la fase de visualización.

- Tratamiento de errores en el scraper.
- Actualización de infraestructura (Docker, workflow (scraper.yml) y requirements.txt).

- Refactorización del esquema de la tabla pois e integración de relación de categorías por ID.
- Implementación de arquitectura de configuración externa y desacoplamiento de parámetros.
- Implementación de sistema de trazabilidad y gestión de logs mediante el módulo logging.

Sprint 8: Extensión del bloque de extracción: investigación de cambios en Maps y de alternativas de extracción, pruebas de ejecución y pulido del sistema (20 de mayo - 2 de junio)

Un contratiempo inesperado en el comportamiento del scraper, provocado por una actualización reciente en el algoritmo de resultados de Google Maps, condicionó el desarrollo de este periodo de trabajo. Debido a este cambio externo que alteraba la precisión de las búsquedas, la mayor parte del esfuerzo se redirigió a investigar, analizar y probar nuevas estrategias de barrido híbridas en entornos de diferente densidad demográfica. En paralelo, se continuó con el perfeccionamiento de la estructura general del sistema, optimizando las tablas de BigQuery en función de los resultados observados y mejorando tanto la arquitectura de archivos de configuración como la gestión del logger. El objetivo final fue dar por concluido el bloque de extracción, procesamiento y automatización, asegurando la operatividad completa del sistema antes de iniciar la fase de visualización y documentación técnica.

- Implementación de filtro por código postal para descartar falsos positivos de Google Maps.
- Pruebas de rendimiento, cobertura y análisis del algoritmo de Google Maps.
- Creación del paquete de inicialización de la base de datos (SQL y CSVs).
- Creación de nueva carpeta de configuración en la raíz del repositorio.
- Implementación de un sistema de historial de archivos de log por ejecución.
- Optimización del diseño visual del logger.

Sprint 9: Conexión con Looker Studio, vistas analíticas y documentación técnica (3 de junio - 16 de junio)

Este sprint se centró en el aprendizaje de la utilización de Looker Studio y el inicio de la documentación técnica del sistema. Se realizaron las primeras pruebas de diseño y selección de gráficos en Looker Studio, estableciendo la conexión con la base de datos cloud mediante la creación de vistas optimizadas en BigQuery con los *joins* ya procesados. En el ámbito documental, se avanzó en la redacción del Apéndice D, correspondiente al manual de programación y la estructura de archivos. Además, durante la elaboración del manual de instalación y ejecución, se detectaron y corrigieron aspectos de la puesta a punto en Docker, optimizando la persistencia de los logs y actualizando las dependencias del contenedor.

- Desarrollo e implementación integral del cuadro de mando en Looker Studio (*Issue general - Fase de inicio y pruebas*).
- Redacción del Apéndice D de los Anexos (Documentación técnica de programación).
- Implementación de vistas en BigQuery y conexión con Looker Studio.
- Configurar persistencia de logs en Docker, actualizar dependencias y validar ejecución local.

Sprint 10: Continuación de Looker Studio y redacción de anexos (17 de junio - 30 de junio)

Este sprint dio continuidad al desarrollo del cuadro de mando en Looker Studio y se focalizó en la redacción de los bloques documentales de los anexos. Durante este periodo, se corrigieron fallos lógicos en el pipeline relacionados con el mapeo dinámico de los atributos de accesibilidad y público infantil, además de la corrección en la asignación de género de los autores de las reseñas. En la documentación, se avanzó en la redacción de los apéndices correspondientes a la planificación temporal, la especificación de requisitos, el diseño del sistema y la sostenibilidad curricular.

- Desarrollo e implementación integral del cuadro de mando en Looker Studio (*Fase de optimización y pruebas avanzadas*).
- Corregir falsos negativos y reestructurar el mapeo dinámico de atributos de accesibilidad y público infantil.

- Corregir el error de identificación de género femenino en las reseñas.
- Documentación del Anexo A: Plan de proyecto.
- Redacción del Apéndice F: Anexo de sostenibilización curricular.
- Apéndice B: Especificación de Requisitos y actualización del diagrama de arquitectura general del proyecto.
- Redacción del apéndice C: Especificación de diseño.

Sprint 11: Redacción de la memoria y preparación de la entrega (1 de julio - 7 de julio)

El último sprint se dedicó al cierre definitivo del dashboard y a la redacción integral de la memoria del TFG y del anexo correspondiente al manual de usuario de Looker Studio. Asimismo, se reestructuró y actualizó el manual técnico del archivo README principal en el repositorio remoto con el objetivo de detallar los pasos de instalación local y automatización del sistema, organizando y subiendo todos los recursos para dejar la entrega final preparada.

- Redacción de la Memoria del TFG.
- Actualizar el manual técnico del archivo README principal.
- Redacción del Apéndice E: Documentación de usuario.

A.3. Estudio de viabilidad

El propósito de este apartado es informar sobre la viabilidad económica y legal del sistema, detallando los costes reales asociados a su puesta en marcha y mantenimiento a largo plazo, así como el cumplimiento del marco jurídico actual.

Viabilidad económica

Para evaluar la viabilidad económica se ha analizado lo que supondría mantener este sistema en producción a lo largo del tiempo. Es importante destacar que los costes de los servicios en la nube están en continuo cambio por parte de los proveedores, por lo que las estimaciones basadas en las pruebas realizadas son orientativas y no fijas. De hecho, en las numerosas

pruebas que se han realizado a lo largo del trabajo, los resultados obtenidos tanto de costes como de tiempo de ejecución varían considerablemente.

A nivel de infraestructura, el coste de las herramientas utilizadas es el siguiente:

- **Google BigQuery:** El almacenamiento y la consulta de datos se mantienen en 0€ al mes gracias a su nivel gratuito permanente, el cual incluye 10 GB de almacenamiento mensual y 1 TB de consultas procesadas al mes [7]. Dado que el volumen de datos de los municipios y puntos de interés (POIs) de la provincia de Burgos es puramente de texto, la base de datos completa de miles de registros ocupa apenas unas decenas de megabytes, quedando totalmente blindada dentro del límite gratuito sin riesgo de generar costes.
- **GitHub Actions y Docker:** La versión gratuita de GitHub Actions impone una restricción crítica, un tiempo máximo de ejecución de 6 horas por cada trabajo independiente [6]. Debido a que la extracción masiva de toda la provincia requiere un tiempo de cómputo estimado de aproximadamente 250 horas según los últimos cálculos, el uso del entorno gratuito estándar de GitHub Actions es inviable para ejecuciones del pipeline completo, limitando su uso a pruebas cortas o requiriendo fragmentar la ejecución en tareas mucho más pequeñas por municipios. En cualquier caso, se podrían realizar las ejecuciones completas conectando un servidor propio al repositorio de GitHub o de forma manual, ya sea a través de Docker o ejecutando el script en la consola.
- **Plataforma de extracción (Apify):** En base a las pruebas reales de consumo de créditos, se estima que una ejecución completa de la provincia cuesta aproximadamente **400\$**. Este cálculo aproximado surge de sumar dos tareas independientes:
 - **Extracción de POIs:** Un coste de ejecución de unos 340\$ para realizar el barrido por municipios y completar la base total de establecimientos o puntos de interés de la provincia de Burgos.
 - **Extracción de reseñas:** Un coste de unos 60\$ basado en una estimación de 50.000 POIs aproximados que puede haber en la provincia según las estimaciones del proyecto y que, por las pruebas realizadas, generen una media de 2 reseñas nuevas al mes, con un coste fijo por reseña procesada de 0,0006\$ según el actor de Apify que se encarga de extraerlas, más un coste residual

de ejecución del actor de unos 2,50\$ en total sumando todas las ejecuciones.

El coste puede variar enormemente en función de posibles actualizaciones tanto en el algoritmo de Google Maps como en los actores de Apify que se encargan de extraer la información. También se ha detectado bastante diferencia especialmente en tiempo de ejecución (aumenta el coste aunque prácticamente no tiene influencia) de unos días a otros o incluso al realizar las pruebas por el día o por la noche, siendo hasta 4 o 5 veces más rápido en algunas ocasiones en comparación a otras, realizando exactamente la misma consulta a Apify.

Al ser un sistema bajo demanda, el coste anual dependerá exclusivamente de la frecuencia de actualización que decida la institución. Para gestionar este coste, se deben tener en cuenta los planes comerciales disponibles en Apify [2]. La plataforma ofrece un descuento del 50 % para estudiantes y fines académicos con una duración máxima de 4 meses (un semestre) vinculando un correo institucional como el de la Universidad de Burgos (@ubu.es) [1]. No obstante, si la UBU decidiese ejecutar el proyecto de forma institucional y permanente, **este descuento del 50 % de estudiante ya no se aplicaría**, debiendo pagar las tarifas comerciales estándar o negociar un contrato personalizado independiente.

En las tarifas, los \$ que pagas son los \$ de créditos que tienes derecho a consumir ese mes (por ejemplo, el Plan Scale: 199\$ de coste al mes -> 199\$ en créditos para consumir al mes). El plan Free ofrece 5\$ mensuales de forma gratuita. Además, hay un descuento progresivo de coste por compute unit a medida que se mejora el plan de suscripción, lo que reduciría el coste ligeramente al contratar uno de los planes superiores. A continuación, en la tabla A.1 se detalla la estructura de planes oficiales de la plataforma (con una conversión de 1 USD = 0,92 EUR):

Plan	Mensual	Pago Anual	Dto. Academ.	Dto. + Anual
Free	0\$ (0€)	0\$ (0€)	0\$ (0€)	0\$ (0€)
Starter	29\$ (26,7€)	26,1\$ (24€)	14,5\$ (13,3€)	13\$ (12€)
Scale	199\$ (183€)	179\$ (164,8€)	99,5\$ (91,5€)	89,5\$ (82,4€)
Business	999\$ (919€)	899\$ (827€)	499,5\$ (459,5€)	449,5\$ (413,6€)
Custom	A consultar	A consultar	No aplicable	No aplicable

Tabla A.1: Planes de precios de Apify (Formato: USD / EUR)

Conclusión y estrategia de explotación económica

El funcionamiento financiero de Apify se basa en un modelo de créditos mensuales que caducan al terminar el ciclo de facturación; es decir, los créditos asignados que no se consumen en el mes corriente se pierden y no se acumulan para el siguiente mes.

Dado que el coste estimado de una actualización global de la provincia es de unos 400\$ de ejecución y que esta tarea no requiere ser ejecutada todos los meses (puesto que los datos de los puntos de interés son generalmente estables), contratar un plan fijo mensual normal como el *Scale* (199\$) sería insuficiente para una carga completa, mientras que el plan *Business* (999\$) generaría un gasto excesivo con créditos desperdiciados en los meses de inactividad del scraper.

Por tanto, la estrategia ideal de explotación económica para la institución consiste en dos opciones:

1. Intentar acordar un **Plan Custom** personalizado mediante un pago anual que asigne una bolsa de créditos global sin caducidad mensual, permitiendo consumir los créditos únicamente durante las campañas específicas de actualización del boletín.
2. Adoptar una estrategia bajo demanda con planes estándar, que consiste en dar de alta el Plan Business (999\$) (o 3 cuentas con un plan Scale para asegurar que cubran todos los costes) de forma puntual únicamente durante el mes en el que se decida realizar la actualización masiva de la provincia, y dar de baja la suscripción durante los meses de descanso del sistema.

Viabilidad legal

El marco legal de este proyecto se evalúa desde tres perspectivas fundamentales: la legitimidad de la extracción automatizada de datos públicos, el cumplimiento de la normativa de protección de datos personales y la gestión de las licencias del software utilizado.

Extracción de datos y propiedad intelectual

Respecto al *web scraping* sobre Google Maps, es necesario señalar que las condiciones de servicio de la plataforma restringen la extracción automatizada de sus datos. Sin embargo, desde el punto de vista del derecho de propiedad intelectual, el sistema se ampara en la **Directiva (UE) 2019/790**

sobre los derechos de autor en el mercado único digital [20]. Esta normativa introduce en sus artículos 3 y 4 una excepción legal para la **Minería de Textos y Datos (TDM)**, que permite la reproducción y extracción de obras de acceso legítimo en internet con fines de investigación y análisis estadístico, siempre que no tengan un propósito comercial y los titulares no hayan restringido explícitamente el acceso mediante archivos de exclusión como *robots.txt*.

Por otra parte, la información estructural relativa a los municipios de la provincia de Burgos y sus cifras oficiales de población provienen del **Instituto Nacional de Estadística (INE)** [9]. El identificador único utilizado en el sistema (*id_municipality*) se corresponde exactamente con el Código de Municipio (CMUN) establecido por dicho organismo, garantizando el cumplimiento de los estándares nacionales de interoperabilidad. Asimismo, la vinculación geográfica de los códigos postales se asienta sobre un conjunto de datos consolidado de acceso abierto que mapea las correspondencias oficiales del INE y la Sociedad Estatal Correos y Telégrafos [5]. Al tratarse de registros administrativos de carácter público y libremente reutilizables para fines de análisis, su incorporación en el pipeline cuenta con plena validez legal.

Dado que el pipeline realiza peticiones espaciadas que respetan la estabilidad de los servidores, no almacena código propietario y solo recopila datos de interés público (coordenadas, nombres comerciales y textos de opiniones), el riesgo de infracción legal es mínimo para un entorno analítico institucional.

Protección de datos personales (RGPD y LOPDGDD)

El tratamiento de los nombres de usuario y los textos de las reseñas se rige por el Reglamento General de Protección de Datos (**RGPD**) **Reglamento (UE) 2016/679** [19] y la ley española **LOPDGDD 3/2018** [10]. El proyecto cumple rigurosamente estas normativas mediante los siguientes principios de diseño:

- **Interés legítimo y fines estadísticos:** El procesamiento se fundamenta en el **Artículo 6.1.f del RGPD**, que ampara el tratamiento necesario para la satisfacción de intereses legítimos (en este caso, la digitalización de la información turística de la provincia de Burgos), en combinación con el **Artículo 89 del RGPD**, el cual permite el tratamiento de datos personales con fines puramente estadísticos y de investigación.

- **Minimización y disociación de datos:** El pipeline aplica el principio de minimización. No se extraen ni almacenan datos críticos como las fotografías de los perfiles ni los enlaces a los perfiles personales. El nombre de usuario que acompaña a la reseña se trata de forma disociada y puramente de texto, considerando que, en muchos casos, se trata de seudónimos públicos.
- **Tratamiento agregado en la visualización:** En la herramienta Looker Studio no se realiza ningún tipo de perfilado individualizado ni toma de decisiones automatizadas sobre personas físicas. Si se realiza alguna estimación (como intentar identificar el género según el nombre de pila), se hace únicamente para obtener estadísticas agrupadas (por ejemplo, ver el porcentaje general de opiniones de hombres o mujeres en un monumento). En ningún momento se hacen perfiles individuales de personas reales.

Gestión de licencias de software

Para garantizar la sostenibilidad del proyecto, se ha analizado la compatibilidad de las licencias del software integrado. El pipeline está construido utilizando componentes de código abierto con licencias permisivas que no imponen restricciones al producto final:

- **Entorno de ejecución y dependencias:** El núcleo del sistema utiliza el lenguaje Python y librerías especializadas bajo licencias libres consolidadas en el sector tecnológico. Específicamente, los módulos de procesamiento de texto y traducción (*pysentimiento* y *deep-translator*) operan bajo la licencia MIT [17, 15], mientras que los componentes críticos de infraestructura como el cliente oficial de Apify (*apify-client*), la autenticación y los conectores cloud (*google-cloud-bigquery*, *google-auth*) están sujetos a la licencia Apache 2.0 [3, 8]. Asimismo, las dependencias de análisis de datos y utilidades (*scikit-learn* y *python-dotenv*) utilizan la licencia BSD de 3 cláusulas [18]. Estas licencias permiten el uso, la modificación y la distribución del software sin generar incompatibilidades legales con el producto desarrollado.
- **Licencia del proyecto:** El código fuente desarrollado en este Trabajo de Fin de Grado se libera bajo la **Licencia MIT**. Esta decisión asegura que la Universidad de Burgos o cualquier entidad pública interesada en el Boletín de Turismo pueda utilizar, auditar y mantener el sistema en el tiempo de forma completamente gratuita y legal, manteniendo la propiedad intelectual del autor original.

Apéndice B

Especificación de Requisitos

B.1. Introducción

Este anexo describe en detalle la estructura de requisitos y el comportamiento funcional del sistema desarrollado para el Boletín de Turismo de la provincia de Burgos. En las siguientes secciones se acotan los objetivos generales del proyecto, se desglosa el catálogo de requisitos funcionales y no funcionales que rigen el comportamiento de la infraestructura, y se especifican los casos de uso principales que describen la interacción de los componentes del pipeline de datos y los usuarios finales con el sistema.

B.2. Objetivos generales

Se definen los siguientes objetivos generales a nivel de ingeniería de software, los cuales cubren el ciclo de vida completo del dato desde su captura inicial hasta su explotación final:

- **OBJ-1 (Automatización de la extracción):** Automatizar la recopilación de puntos de interés turísticos y opiniones de usuarios para los municipios de la provincia de Burgos, eliminando la necesidad de realizar búsquedas manuales por parte del personal institucional.
- **OBJ-2 (Procesamiento y enriquecimiento):** Desarrollar un módulo de procesamiento capaz de limpiar los datos en bruto extraídos, traducir automáticamente las reseñas internacionales al castellano y clasificar el sentimiento de las opiniones mediante modelos de lenguaje para aportar valor estadístico.

- **OBJ-3 (Centralización del almacenamiento):** Centralizar el almacenamiento de toda la información recogida en una base de datos estructurada, segura y de bajo coste operativo en la nube, garantizando la integridad de los datos mediante el uso de identificadores e información base necesaria extraída de lugares oficiales (INE).
- **OBJ-4 (Explotación analítica y visualización):** Diseñar un cuadro de mando interactivo y visual en Looker Studio que permita a los gestores institucionales del Boletín de Turismo monitorizar, filtrar y analizar fácilmente las tendencias, valoraciones y la percepción de los visitantes en la provincia.
- **OBJ-5 (Integración y autonomía del pipeline):** Integrar todas las fases del proceso (extracción, transformación y carga) en un pipeline único, portable y contenedorizado que permita actualizar toda la información del sistema bajo demanda de manera completamente autónoma.

B.3. Catálogo de requisitos

Requisitos funcionales

RF-1 Configurar el sistema

El sistema debe permitir parametrizar el comportamiento del pipeline de forma externa a través de un archivo de configuración.

- **RF-1.1 Configurar márgenes de actualización:** El sistema debe permitir fijar los días de caducidad para volver a extraer datos y reseñas.
- **RF-1.2 Configurar filtros de ejecución:** El sistema debe permitir limitar el proceso a un único municipio, categoría o punto de interés concreto.
- **RF-1.3 Configurar criterios de ordenación:** El sistema debe permitir definir la prioridad de recorrido de los bucles mediante ordenaciones SQL.
- **RF-1.4 Configurar límites demográficos:** El sistema debe permitir establecer los límites de habitantes para clasificar los municipios en rural, semi-rural y urbano.

- **RF-1.5 Configurar el nivel de detalle de los logs:** El sistema debe permitir definir la precisión de las trazas de ejecución.

RF-2 Controlar la ejecución secuencial del pipeline

El sistema debe permitir su ejecución desatendida y el control del flujo del programa.

- **RF-2.1 Ejecutar de forma programada:** El sistema debe iniciarse automáticamente de manera periódica a través de GitHub Actions según el horario configurado.
- **RF-2.2 Ejecutar de forma manual:** El sistema debe permitir que un usuario active el pipeline bajo demanda desde la plataforma de desarrollo.

RF-3 Extraer información

El sistema debe buscar y extraer datos oficiales de Google Maps de forma automatizada.

- **RF-3.1 Recorrer municipios en bucle:** El sistema debe recorrer secuencialmente la lista oficial de municipios de Burgos para organizar la ejecución.
- **RF-3.2 Variar la extracción según la clasificación del municipio:** El sistema debe adaptar la estrategia de búsqueda basándose en el volumen de población del municipio.
- **RF-3.3 Recorrer categorías en bucle:** El sistema debe realizar búsquedas ordenadas dentro de cada municipio por cada categoría almacenada en la base de datos.
- **RF-3.4 Extraer puntos de interés:** El sistema debe recopilar la información de lugares y establecimientos de la provincia de Burgos.
- **RF-3.5 Filtrar por código postal:** El sistema debe comprobar y descartar los puntos de interés que queden fuera del término municipal consultado.
- **RF-3.6 Extraer reseñas:** El sistema debe descargar las reseñas y valoraciones asociadas a cada punto de interés extraído.

RF-4 Procesar y limpiar reseñas

El sistema debe transformar y enriquecer las reseñas obtenidas antes de guardarlas en la base de datos.

- **RF-4.1 Eliminar reseñas sin valor:** El sistema debe descartar las reseñas breves o sin texto que no aporten valor analítico.
- **RF-4.2 Traducir reseñas:** El sistema debe traducir al idioma solicitado en la configuración los textos de las reseñas que se hayan escrito en otros idiomas.
- **RF-4.3 Detectar el género:** El sistema debe estimar de forma agrupada el género del autor a partir de su nombre.
- **RF-4.4 Analizar el sentimiento:** El sistema debe evaluar el texto para asignar una nota de satisfacción en un rango entre -1 y 1.

RF-5 Almacenar y optimizar

El sistema debe gestionar la base de datos relacional optimizando los costes y los recursos informáticos.

- **RF-5.1 Evitar descargas redundantes:** El sistema debe parar la descarga de reseñas al encontrar una reseña ya almacenada.
- **RF-5.2 Calcular el TORI:** El sistema debe combinar las estrellas de Maps y el sentimiento calculado en una nota final escalada desde la configuración.
- **RF-5.3 Gestionar categorías nuevas:** El sistema debe almacenar las categorías oficiales de Google Maps de los POIs y si se extraen por primera vez clasificarlas dentro de los niveles de categorías.
- **RF-5.4 Cargar datos en BigQuery:** El sistema debe usar tablas temporales para actualizar o añadir registros nuevos sin duplicados.
- **RF-5.5 Limpiar tablas temporales:** El sistema debe eliminar las tablas temporales de almacenamiento intermedio una vez que se haya completado la subida de datos a BigQuery.
- **RF-5.6 Registrar marcas de control:** El sistema debe apuntar el momento de finalización exitosa de extracción por municipio, categoría y POI.

RF-6 Monitorizar la ejecución

El sistema debe encargarse de la monitorización operativa y el registro de la actividad del pipeline.

- **RF-6.1 Mostrar logs de ejecución:** El sistema debe mostrar las trazas operativas y los errores en la terminal en tiempo real.
- **RF-6.2 Almacenar historial de logs:** El sistema debe generar y guardar un archivo de texto físico independiente con los registros de cada ejecución.

RF-7 Visualizar datos

El sistema debe ofrecer herramientas analíticas visuales para la explotación final de la información recogida.

Requisitos no funcionales

- **RNF-1 Contenedorización:** Todo el entorno de ejecución del pipeline debe estar encapsulado en un contenedor Docker para garantizar su portabilidad.
- **RNF-2 Automatización:** El sistema debe permitir su ejecución de forma programada o desatendida mediante flujos de trabajo de GitHub Actions sin requerir intervención manual constante.
- **RNF-3 Seguridad:** Las credenciales de los servicios cloud de Google y Apify deben estar completamente aisladas del código fuente, gestionándose a través de variables de entorno seguras.
- **RNF-4 Optimización de costes:** El consumo informático derivado de las consultas y el almacenamiento de datos en la nube debe garantizar el menor coste posible a través de filtros de seguridad que eviten el consumo excesivo o no deseado.
- **RNF-5 Tolerancia a fallos:** El pipeline debe capturar las excepciones de red, caídas de las APIs externas o inconsistencias de datos estructurados de un punto de interés concreto mediante bloques *try-except*, forzando la continuación del bucle hacia el siguiente local de la lista para impedir la interrupción total de la ejecución global.

- **RNF-6 Interoperabilidad administrativa:** La base de datos debe estructurarse utilizando identificadores y códigos estándar nacionales para permitir que los datos recogidos sean compatibles con otros sistemas de la administración pública.

B.4. Especificación de requisitos

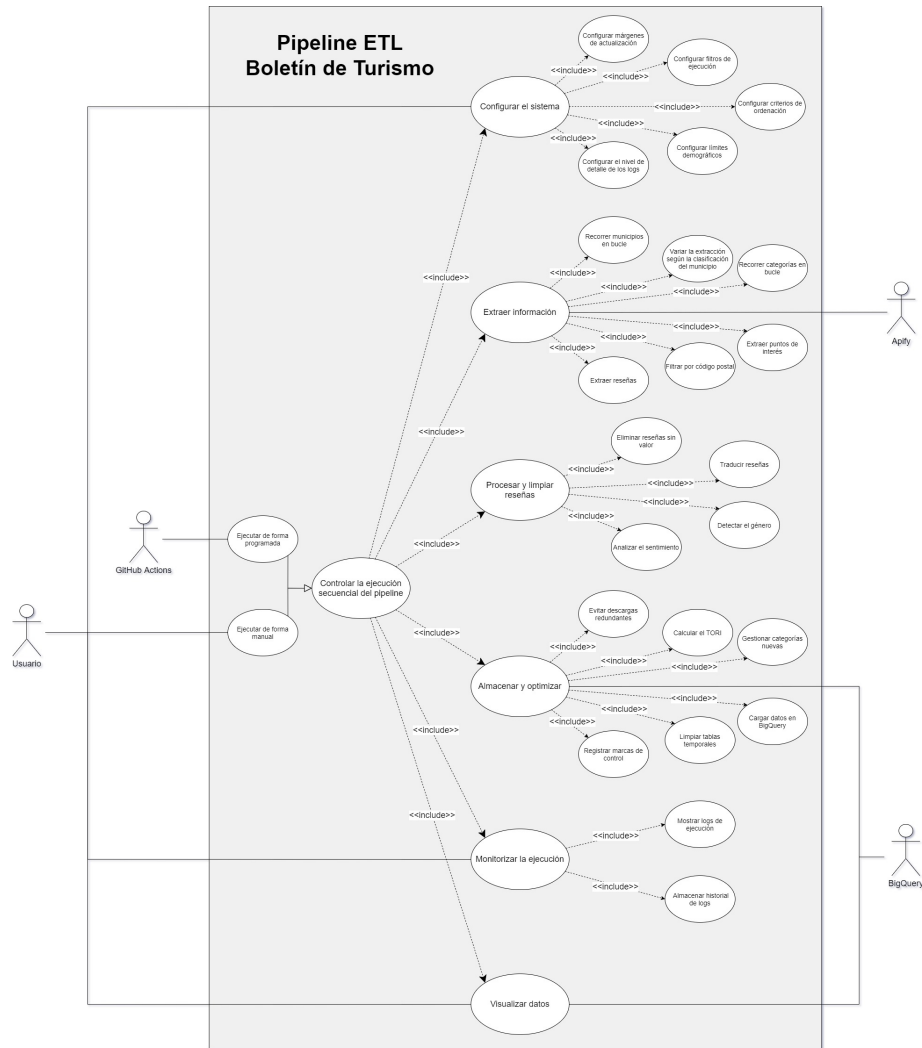


Figura B.1: Diagrama completo de casos de uso del pipeline ETL.

CU-1	Configurar el sistema
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-1, RF-1.1, RF-1.2, RF-1.3, RF-1.4, RF-1.5
Descripción	Permite al usuario definir las reglas de negocio, los filtros de búsqueda y las opciones de control del programa modificando el archivo de configuración de forma externa.
Precondición	El archivo <code>config/config.json</code> debe existir en el directorio del proyecto y estar correctamente estructurado.
Acciones	1. El usuario abre el archivo de configuración <code>config.json</code>. 2. El usuario modifica los parámetros de configuración de los márgenes, filtros, ordenaciones, límites demográficos, etc. según sus necesidades. 3. El usuario guarda los cambios en el archivo. 4. Al arrancar el programa, el sistema lee este archivo modificado y aplica los nuevos cambios a toda la ejecución.
Postcondición	El comportamiento del programa queda adaptado a las nuevas preferencias guardadas por el usuario.
Excepciones	Si el usuario comete un error al escribir el formato del parámetro o borra campos obligatorios, el programa fallará al arrancar, mostrará un error y se detendrá.
Importancia	Alta

Tabla B.1: CU-1 Configurar el sistema.

CU-1.1	Configurar márgenes de actualización
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-1.1
Descripción	El usuario define cuántos días deben pasar antes de que el programa considere que los POIs de un municipio o categoría o de sus reseñas necesiten actualizarse o extraer nuevos otra vez.
Precondición	El archivo <code>config/config.json</code> debe existir en el directorio del proyecto y estar correctamente estructurado.
Acciones	1. El usuario busca las variables de margen de actualización en el archivo. 2. El usuario escribe el número de días límite deseado para los puntos de interés y para las reseñas. 3. El usuario guarda el archivo.
Postcondición	Los nuevos días de margen quedan fijados y el código los usará para decidir si actualiza un sitio o si ya se ha procesado hace menos días de los indicados y lo salta.
Excepciones	Si el usuario comete un error al escribir el formato del parámetro o borra campos obligatorios, el programa fallará al arrancar, mostrará un error y se detendrá.
Importancia	Alta

Tabla B.2: CU-1.1 Configurar márgenes de actualización.

CU-1.2	Configurar filtros de ejecución
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-1.2
Descripción	El usuario escribe un filtro específico para obligar al programa a procesar un único municipio, categoría o POI concreto, en lugar de analizar toda la provincia.
Precondición	El archivo <code>config/config.json</code> debe existir en el directorio del proyecto y estar correctamente estructurado. El elemento indicado para filtrar debe estar guardado exactamente con ese nombre en la base de datos.
Acciones	1. El usuario busca las variables de filtrado en el archivo. 2. El usuario escribe el nombre exacto del municipio, la categoría o el sitio que quiere probar o actualizar de forma aislada o deja los tres en blanco para hacer la ejecución completa. 3. El usuario guarda los cambios.
Postcondición	El programa limitará su rango de búsqueda únicamente al elemento escrito por el usuario en su próximo arranque o si se indica el nombre del POI, solo sacará reseñas de ese POI sin hacer extracción de lugares.
Excepciones	Ninguna. Si el usuario escribe un nombre que no existe, el programa simplemente no encontrará datos y terminará de forma limpia.
Importancia	Media

Tabla B.3: CU-1.2 Configurar filtros de ejecución.

CU-1.3	Configurar criterios de ordenación
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-1.3
Descripción	El usuario define la prioridad y el orden en el que el programa debe recorrer las listas de municipios o categorías al pedirlos a la base de datos.
Precondición	El archivo <code>config/config.json</code> debe existir en el directorio del proyecto y estar correctamente estructurado.
Acciones	1. El usuario busca el apartado de ordenación en el archivo. 2. El usuario escribe el criterio de ordenación en formato SQL (por ejemplo, ordenar por número de habitantes de mayor a menor: <code>population DESC</code>). 3. El usuario guarda el archivo.
Postcondición	El programa inyectará este orden en sus consultas para recorrer los bucles en la secuencia elegida por el usuario.
Excepciones	Si el usuario escribe mal el nombre de la columna, la base de datos dará un error de formato al arrancar el programa y este se detendrá.
Importancia	Baja

Tabla B.4: CU-1.3 Configurar criterios de ordenación.

CU-1.4	Configurar límites demográficos
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-1.4
Descripción	El usuario fija los umbrales de habitantes necesarios para que el programa clasifique automáticamente cada municipio como entorno rural, semi-rural o urbano.
Precondición	El archivo <code>config/config.json</code> debe existir en el directorio del proyecto y estar correctamente estructurado.
Acciones	1. El usuario busca las variables de límite de población en el archivo <code>config.json</code>. 2. El usuario escribe el número de habitantes que marca la frontera entre un tipo de municipio y otro. 3. El usuario guarda los cambios.
Postcondición	El programa usará estos números guardados para asignar un nivel a los municipio y cambiar la estrategia de búsqueda según el nivel de cada uno.
Excepciones	Ninguna.
Importancia	Alta

Tabla B.5: CU-1.4 Configurar límites demográficos.

CU-1.5	Configurar el nivel de detalle de los logs
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-1.5
Descripción	El usuario selecciona cuánta información quiere ver por pantalla durante la ejecución (solo errores, o también avisos y detalles técnicos) y si desea activar el guardado en un archivo físico.
Precondición	El archivo <code>config/config.json</code> debe existir en el directorio del proyecto y estar correctamente estructurado.
Acciones	1. El usuario busca el bloque de configuración del sistema de mensajes. 2. El usuario escribe el nivel de detalle que quiere monitorizar (como DEBUG o INFO). 3. El usuario activa o desactiva la opción booleana para clonar estos mensajes en un archivo de texto en el repositorio. 4. El usuario guarda el archivo.
Postcondición	El sistema de logs adaptará los mensajes de la terminal y los archivos físicos a las preferencias del usuario.
Excepciones	Ninguna. Si el usuario escribe mal el nombre del nivel, el programa utilizará el modo INFO por defecto para evitar romperse.
Importancia	Baja

Tabla B.6: CU-1.5 Configurar el nivel de detalle de los logs.

CU-2	Controlar la ejecución secuencial del pipeline
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-2, RF-2.1, RF-2.2
Descripción	El componente central del código (main_scraper_pipeline.py) se encarga de gestionar el ciclo de vida del programa, asegurando que los módulos se activen en el orden correcto y controlando la sesión.
Precondición	El entorno de ejecución está listo junto con los tokens de Apify y GCP, y el archivo de configuración está accesible.
Acciones	1. El sistema recibe la orden de inicio desde uno de los disparadores configurados. 2. El archivo principal toma el control del sistema operativo y levanta el proceso en memoria. 3. El programa coordina el paso de datos entre los distintos módulos, vigilando que ninguno rompa la secuencia. 4. El programa detecta la finalización de los bucles de trabajo y cierra el proceso de forma segura.
Postcondición	El pipeline se apaga correctamente tras completar todas las tareas secuenciales del ciclo.
Excepciones	Si ocurre un fallo crítico imprevisto en el sistema que impida la comunicación entre módulos, el programa aborta la sesión actual para evitar corromper las bases de datos.
Importancia	Crítica

Tabla B.7: CU-2 Controlar la ejecución secuencial del pipeline.

CU-2.1	Ejecutar de forma programada
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-2.1
Descripción	La plataforma GitHub Actions arranca el programa de forma automática y desatendida cada cierto tiempo siguiendo un horario programado.
Precondición	El workflow (archivo scraper.yml) está configurado y activo en el repositorio de GitHub con un cron correcto. Están configurados los tokens de Apify y GCP en GitHub Secrets.
Acciones	1. El reloj interno de GitHub Actions detecta que ha llegado la hora programada en el cron del workflow. 2. GitHub Actions levanta un entorno virtual y descarga el código del proyecto. 3. Se lanza el comando automático que inicia la ejecución del programa.
Postcondición	El programa principal se pone en marcha de forma automática sin necesidad de que intervenga ningún usuario.
Excepciones	Si los servidores de GitHub están caído, el cron es incorrecto o no están las claves en los secretos, la ejecución no llegará a iniciarse.
Importancia	Baja

Tabla B.8: CU-2.1 Ejecutar de forma programada.

CU-2.2	Ejecutar de forma manual
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-2.2
Descripción	Permite al lanzar el programa de forma inmediata y bajo demanda en una terminal.
Precondición	El usuario tiene acceso local o remoto al repositorio, contenedor Docker o terminal donde está instalado el proyecto.
Acciones	1. El usuario abre la terminal de comandos del sistema operativo (o la consola de Docker Desktop). 2. El usuario introduce el comando de arranque manual del programa principal. 3. El programa ejecuta todas sus funciones de manera automática sin necesidad de mayor intervención del usuario.
Postcondición	El pipeline comienza a ejecutarse en tiempo real bajo la supervisión directa del usuario en la terminal.
Excepciones	Si el usuario introduce mal el comando o no tiene permisos de administrador en la consola, el sistema operativo deniega el arranque y no se inicia el pipeline.
Importancia	Alta

Tabla B.9: CU-2.2 Ejecutar de forma manual.

CU-3	Extraer información
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-3, RF-3.1, RF-3.2, RF-3.3, RF-3.4, RF-3.5, RF-3.6
Descripción	El sistema se conecta con la API de Apify para buscar y descargar los datos de los sitios turísticos de Burgos y sus correspondientes reseñas.
Precondición	El token de acceso a la API de Apify está guardado y es válido.
Acciones	1. El programa pide a BigQuery la lista de municipios y las categorías turísticas que debe procesar. 2. El programa clasifica cada municipio y organiza la estrategia de búsqueda. 3. El programa llama al actor extractor de Apify para descargar la información de los sitios encontrados. 4. El programa limpia los resultados incorrectos y aplica el filtro por código postal. 5. El programa llama al actor de reseñas de Apify para descargar los comentarios de cada sitio válido.
Postcondición	Los datos de los sitios y sus comentarios quedan descargados en la memoria del programa.
Excepciones	Si la API de Apify no responde o da un error, el programa reintenta la conexión un número máximo configurado de veces antes de detenerse con un fallo.
Importancia	Alta

Tabla B.10: CU-3 Extraer información.

CU-3.1	Recorrer municipios en bucle
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-3.1
Descripción	El programa lee la lista de municipios de la provincia de Burgos desde BigQuery y va procesando uno por uno de forma ordenada.
Precondición	Existe una tabla en BigQuery que contiene la lista oficial de municipios.
Acciones	1. El programa lanza una consulta a BigQuery para traer los municipios pendientes de actualización. 2. El programa ordena la lista según el criterio configurado. 3. El programa abre un bucle para procesar el primer municipio de la lista y, al acabar, pasa automáticamente al siguiente.
Postcondición	Cada municipio pendiente es seleccionado secuencialmente para extraer sus datos.
Excepciones	Si no encuentra la tabla de municipios da error y si la base de datos devuelve una lista vacía porque todo está actualizado, el programa registra un aviso y termina sin hacer nada.
Importancia	Alta

Tabla B.11: CU-3.1 Recorrer municipios en bucle.

CU-3.2	Variar la extracción según la clasificación del municipio
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-3.2
Descripción	El programa comprueba los habitantes del municipio y adapta los parámetros de búsqueda de Apify según sea un entorno rural, semi-rural o urbano.
Precondición	1. El municipio que se está procesando tiene asignado su número de habitantes. 2. Se indican los límites de cada nivel de municipio en el archivo de configuración
Acciones	1. El programa comprueba si el número de habitantes es menor que el indicado en la configuración como máximo del nivel rural, después si es menor que el valor indicado como límite de semi-rural o si es mayor y por tanto, urbano. 2. Si es un municipio grande (urbano), el programa añade un filtro de localización geográfico estricto al input de Apify para forzar una búsqueda exhaustiva. 3. Si es un municipio pequeño, simplifica los parámetros para ahorrar costes de computación en la API y recortar el tiempo empleado en la extracción.
Postcondición	La consulta de búsqueda enviada a Apify cambia de forma dinámica según el tamaño del municipio.
Excepciones	Si no hay datos de población para un municipio, se le asigna el nivel rural.
Importancia	Alta

Tabla B.12: CU-3.2 Variar la extracción según la clasificación del municipio.

CU-3.3	Recorrer categorías en bucle
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-3.3
Descripción	Dentro de cada municipio, el programa abre otro bucle para buscar uno por uno los sitios que pertenecen a cada categoría guardada en la base de datos (hoteles, restaurantes, museos, etc.).
Precondición	El programa tiene cargada en memoria la lista de categorías permitidas para el nivel de ese municipio.
Acciones	1. El programa selecciona la primera categoría de la lista oficial de taxonomías. 2. El programa comprueba en la tabla de control si esa categoría específica ya se ha buscado hace poco en este municipio. 3. Si ya se ha buscado recientemente, el programa se salta la categoría; si no, avanza al paso de extracción de Apify.
Postcondición	El programa organiza las búsquedas de Google Maps combinando el nombre del municipio y la categoría.
Excepciones	Si no encuentra la tabla de categorías da error y si la base de datos devuelve una lista vacía, el programa registra un aviso y termina sin hacer nada al no detectar categorías que necesiten actualización
Importancia	Alta

Tabla B.13: CU-3.3 Recorrer categorías en bucle.

CU-3.4	Extraer puntos de interés
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-3.4
Descripción	El programa llama al actor de Apify para descargar la información de los puntos de interés.
Precondición	1. El programa ha construido la frase de consulta con la categoría y el municipio correspondiente. 2. El token de acceso a la API de Apify está guardado y es válido.
Acciones	1. El programa envía la consulta al actor de Apify en la nube. 2. Apify extrae los sitios encontrados en Google Maps y devuelve un listado en formato JSON. 3. El programa recoge la lista.
Postcondición	Se obtiene una lista limpia de datos con los puntos de interés encontrados.
Excepciones	Si la API de Apify no responde o da un error, el programa reintenta la conexión un número máximo configurado de veces antes de detenerse con un fallo.
Importancia	Alta

Tabla B.14: CU-3.4 Extraer puntos de interés.

CU-3.5	Filtrar por código postal
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-3.5
Descripción	El programa comprueba el código postal de cada sitio descargado y borra los que pertenezcan a otros municipios para evitar errores de geolocalización de Google Maps.
Precondición	El programa tiene cargado el conjunto de códigos postales oficiales autorizados para el municipio actual.
Acciones	1. El programa lee el código postal del sitio devuelto por la API. 2. Si el código postal viene vacío, el programa busca si aparece algún código de cinco dígitos dentro del texto de su dirección completa para rescatarlo. 3. El programa comprueba si el código postal empieza por "09z si pertenece a la lista de códigos autorizados de ese municipio. 4. Si el código postal no está en la lista permitida, el programa descarta el sitio por completo y muestra un aviso por pantalla.
Postcondición	Solo los sitios turísticos que están físicamente dentro del término municipal real se mantienen en la lista de guardado.
Excepciones	Si el municipio no tiene códigos postales guardados en la base de datos, el programa aplica un filtro genérico comprobando que empiece por el prefijo "09" de Burgos.
Importancia	Alta

Tabla B.15: CU-3.5 Filtrar por código postal.

CU-3.6	Extraer reseñas
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-3.6
Descripción	El programa utiliza el identificador único de cada POI guardado en la base de datos para llamar al scraper de reseñas de Apify.
Precondición	El token de acceso a la API de Apify está guardado y es válido.
Acciones	1. El programa construye una URL usando el identificador del punto de interés del que se van a extraer las reseñas (<code>poi_id</code>). 2. Envía la petición a Apify configurando los parámetros para traer las reseñas limitando el tiempo según los parámetros indicados en el archivo de configuración. 3. Recibe el lote completo de reseñas en bruto de ese sitio y los envía al módulo de transformación.
Postcondición	Se descarga el listado de opiniones nuevas de ese establecimiento listas para ser analizadas.
Excepciones	Si el sitio no tiene reseñas registradas en Google Maps, la API devuelve una lista vacía y el programa pasa al siguiente POI de forma limpia.
Importancia	Alta

Tabla B.16: CU-3.6 Extraer reseñas.

CU-4	Procesar y limpiar reseñas
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-4, RF-4.1, RF-4.2, RF-4.3, RF-4.4
Descripción	El programa procesa y transforma cada reseña extraída para guardarla en la base de datos con la información completa.
Precondición	Se ha descargado alguna reseña.
Acciones	1. El programa analiza cada reseña descargada de forma individual. 2. El programa descarta los textos vacíos o demasiado cortos. 3. El programa limpia los textos eliminando formatos molestos y traduce las reseñas de otros idiomas. 4. El programa calcula estadísticamente el género del usuario y la puntuación de sentimiento del texto.
Postcondición	Se obtiene un conjunto de datos limpio y enriquecido listo para guardarse en la base de datos.
Excepciones	Si faltan datos obligatorios, el programa introduce valores por defecto para poder continuar sin romperse.
Importancia	Alta

Tabla B.17: CU-4 Procesar y limpiar reseñas.

CU-4.1	Eliminar reseñas sin valor
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-4.1
Descripción	El programa mide la longitud del comentario y descarta automáticamente los que tengan menos de las palabras indicadas como mínimas en el archivo de configuración o estén completamente vacíos para asegurar la calidad de los datos.
Precondición	La reseña está siendo procesada en el módulo de transformación.
Acciones	1. El programa cuenta el número de palabras que componen el texto de la reseña. 2. El programa compara este número con el límite mínimo configurado en los parámetros de configuración. 3. Si no alcanza el mínimo, el programa descarta la reseña y registra un aviso de depuración.
Postcondición	Solo las reseñas con texto relevante y suficiente longitud continúan en el flujo de guardado.
Excepciones	Ninguna.
Importancia	Alta

Tabla B.18: CU-4.1 Eliminar reseñas sin valor.

CU-4.2	Traducir reseñas
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-4.2
Descripción	El programa borra las etiquetas basura de Google Maps, detecta el idioma real del texto y lo traduce automáticamente al idioma pedido en configuración si se escribió en otra lengua.
Precondición	La reseña ha superado el filtro de longitud mínima de palabras.
Acciones	1. El programa limpia los saltos de línea y borra los textos fijos". 2. Una librería analiza el texto y detecta el idioma real de la reseña de forma independiente. 3. Si el idioma es distinto al idioma objetivo, el programa traduce el comentario.
Postcondición	El texto de la reseña queda traducido y sin textos basura de la interfaz de Google.
Excepciones	Ninguna
Importancia	Alta

Tabla B.19: CU-4.2 Traducir reseñas.

CU-4.3	Detectar el género
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-4.3
Descripción	El programa aísla el primer nombre del autor de la reseña y utiliza una base de datos estadística para estimar si es un usuario masculino o femenino.
Precondición	El nombre del autor de la reseña está disponible en los datos de origen.
Acciones	1. El programa separa el nombre completo del autor y se queda únicamente con la primera palabra. 2. El programa envía este primer nombre al detector automático de género. 3. Si el nombre se asocia a mujer se guarda como "Femenino", si se asocia a hombre se guarda como "Masculino" si es ambiguo se marca con el valor por defecto.
Postcondición	Se añade una nueva columna demográfica de género a la información de la reseña.
Excepciones	Si el detector de nombres falla o el nombre no se encuentra en el diccionario, el programa le asigna el valor establecido por defecto de forma controlada.
Importancia	Baja

Tabla B.20: CU-4.3 Detectar el género.

CU-4.4	Analizar el sentimiento
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-4.4
Descripción	El programa analiza el texto traducido mediante un modelo de lenguaje para clasificar la opinión en una categoría y calcular una nota numérica de satisfacción.
Precondición	El comentario de la reseña ya se encuentra limpio y traducido al castellano.
Acciones	1. El programa envía el texto de la reseña al analizador de lenguaje de la librería <code>pysentimiento</code>. 2. El modelo clasifica el comentario de forma interna dentro de una etiqueta fija (Positivo, Neutral o Negativo). 3. El programa resta la probabilidad negativa a la positiva para obtener una nota matemática continua en un rango entre -1 y 1.
Postcondición	La reseña recibe una nota numérica exacta de sentimiento que representa el nivel de satisfacción del autor de la reseña.
Excepciones	Si el modelo de inteligencia artificial da un error inesperado al procesar el texto, el programa le asigna una etiqueta "NEU" una nota de 0.0 para poder continuar.
Importancia	Alta

Tabla B.21: CU-4.4 Analizar el sentimiento.

CU-5	Almacenar y optimizar
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-5, RF-5.1, RF-5.2, RF-5.3, RF-5.4, RF-5.5, RF-5.6
Descripción	El programa coge los lotes de datos que ya están limpios y transformados en la memoria y los guarda de forma definitiva, ordenada y segura en las tablas de Google BigQuery.
Precondición	El programa tiene datos listos para guardar en memoria y la conexión con Google Cloud funciona correctamente.
Acciones	1. El programa establece comunicación con el almacén de datos de Google BigQuery. 2. El programa vuelca los datos de los nuevos puntos de interés y sus reseñas en las tablas correspondientes. 3. El programa recalcula la nota de reputación (índice TORI) de cada POI con los nuevos datos añadidos. 4. El programa registra la fecha y hora actuales en los campos de control de las tablas para marcar que el trabajo se ha completado con éxito. 5. El programa limpia y elimina todos los recursos o tablas temporales que se han usado durante el proceso de subida.
Postcondición	Las tablas de la base de datos quedan actualizadas con la información más reciente y listas para que las consulte Looker Studio.
Excepciones	Si la base de datos da un error al guardar o se corta la conexión, el programa registra el fallo por pantalla, borra obligatoriamente las tablas temporales creadas para no generar costes y se detiene de forma segura.
Importancia	Alta

Tabla B.22: CU-5 Almacenar y optimizar.

CU-5.1	Evitar descargas redundantes
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-5.1
Descripción	El programa frena la descarga de reseñas de un sitio en cuanto encuentra un identificador que ya está guardado en BigQuery, aprovechando que las reseñas vienen ordenadas desde la más nueva.
Precondición	El programa ha consultado el identificador de la reseña más reciente de ese sitio antes de llamar a la API.
Acciones	1. El programa recorre el listado de comentarios que va devolviendo Apify de uno en uno. 2. El programa compara el identificador de la opinión actual con el último guardado en la base de datos. 3. Al detectar una coincidencia exacta, el programa detiene el recorrido del bucle para no procesar el resto de reseñas que seguro que están ya almacenadas al ser más viejas que la última almacenada.
Postcondición	Se evita el procesamiento y la descarga repetida de reseñas, ahorrando costes y tiempo de computación.
Excepciones	Si el POI no tiene reseñas, no se aplica esta funcionalidad y extrae todas las reseñas correspondientes.
Importancia	Media

Tabla B.23: CU-5.1 Evitar descargas redundantes.

CU-5.2	Calcular el TORI
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-5.2
Descripción	El programa combina la puntuación de estrellas de Google Maps y la media del análisis de sentimiento en una nota de reputación final.
Precondición	El POI del que se quiere calcular el TORI tiene reseñas guardadas en base de datos.
Acciones	1. El programa obtiene de la configuración el valor máximo que se le da a cada componente del cálculo (valoración y sentimiento). 2. El programa calcula la media de todas las puntuaciones de sentimiento guardadas para ese establecimiento concreto. 3. Se ejecuta una consulta para actualizar la nota del TORI de ese POI en la base de datos.
Postcondición	El sitio turístico actualiza su nota global de reputación basándose en las nuevas reseñas tratadas.
Excepciones	Si el POI no tiene comentarios con texto en la base de datos, el programa aplica un valor de cero a la media de sentimiento para calcular la nota.
Importancia	Alta

Tabla B.24: CU-5.2 Calcular el TORI.

CU-5.3	Gestionar categorías nuevas
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-5.3
Descripción	Si el programa encuentra una categoría nueva en Google Maps que no estaba registrada, calcula un número de identificación nuevo y la añade automáticamente a la lista de la base de datos.
Precondición	El programa ha extraído un POI que pertenece a una categoría no registrada en la base de datos.
Acciones	1. El programa mira si la categoría ya existe en la base de datos. 2. Si no existe, busca el código numérico más alto en la base de datos y le suma uno para crear el siguiente identificador. 3. El programa guarda la nueva categoría en la tabla de BigQuery para poder usarla a partir de ese momento.
Postcondición	La lista de categorías se actualiza en la base de datos y el POI se guarda bien enlazado.
Excepciones	Si ocurre un error al intentar guardar la nueva categoría en la base de datos, el programa muestra el fallo por pantalla y se salta ese sitio para seguir con los demás.
Importancia	Media

Tabla B.25: CU-5.3 Gestionar categorías nuevas.

CU-5.4	Cargar datos en BigQuery
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-5.4
Descripción	Sube todos los datos limpios juntos a una tabla temporal en internet y luego los junta con la tabla definitiva.
Precondición	Que haya un POI o reseña extraídos
Acciones	1. El programa junta los datos acumulados en la memoria y los sube creando una tabla temporal en la base de datos. 2. El programa pide a BigQuery que compare la tabla temporal con la definitiva para añadir lo nuevo y actualizar lo que haya cambiado.
Postcondición	Los datos de los POIs y las reseñas se guardan de forma definitiva e integrada en la nube.
Excepciones	Si hay un error con el formato de los datos, el programa detiene la subida, avisa por pantalla y lanza un aviso de fallo.
Importancia	Alta

Tabla B.26: CU-5.4 Cargar datos en BigQuery.

CU-5.5	Limpiar tablas temporales
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-5.5
Descripción	El programa borra la tabla temporal de la base de datos en cuanto termina de guardar los datos para no ocupar espacio de más ni generar gastos extra en la nube.
Precondición	El programa ha terminado de lanzar la orden para juntar los datos en la base de datos.
Acciones	1. El programa termina de procesar la subida principal de los datos. 2. Se envía una orden directa a BigQuery para borrar del mapa la tabla temporal que se creó para esa ejecución.
Postcondición	La base de datos queda totalmente limpia de archivos temporales de almacenamiento intermedio.
Excepciones	Si el programa no puede borrar la tabla temporal por un fallo en la conexión, muestra un aviso de advertencia por pantalla y continúa.
Importancia	Media

Tabla B.27: CU-5.5 Limpiar tablas temporales.

CU-5.6	Registrar marcas de control
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-5.6
Descripción	Guarda el día y la hora exactos en los que se termina de actualizar un municipio o categoría para que el programa sepa que está al día en las siguientes ejecuciones.
Precondición	El proceso de descarga e inserción de datos de ese bloque ha terminado correctamente y sin errores pasando por todos los elementos del bucle.
Acciones	1. El programa genera una marca con la hora y la fecha actuales del ordenador. 2. Guarda una fila en la tabla de control apuntando el nombre del municipio, la categoría y la hora actual. 3. Actualiza las fechas de última revisión en las tablas principales de municipios y de POIs.
Postcondición	Quedan guardadas las fechas de control que servirán para saber qué partes están actualizadas en el próximo arranque.
Excepciones	Ninguna.
Importancia	Alta

Tabla B.28: CU-5.6 Registrar marcas de control.

CU-6	Monitorizar la ejecución
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-6, RF-6.1, RF-6.2
Descripción	El sistema muestra la actividad del programa por pantalla en tiempo real y permite elegir, según lo que configure el usuario, si también se desea guardar un archivo de texto en el ordenador.
Precondición	El programa principal está arrancando o ya está funcionando en memoria.
Acciones	1. El sistema empieza a capturar todos los avisos y mensajes que generan las distintas partes del código. 2. El sistema envía siempre los mensajes a la pantalla de la terminal para poder verlos en directo (CU-6.1). 3. El sistema revisa el archivo de configuración y, solo si el usuario ha activado la opción, guarda también esos mensajes en un archivo de texto (CU-6.2).
Postcondición	Toda la actividad técnica se ve en directo por la consola y, de forma opcional, queda salvada en un archivo físico en el disco duro.
Excepciones	Si el sistema de mensajes falla al encenderse, el programa funcionará a ciegas sin mostrar información de lo que hace.
Importancia	Media

Tabla B.29: CU-6 Monitorizar la ejecución.

CU-6.1	Mostrar logs de ejecución
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-6.1
Descripción	Muestra por pantalla en tiempo real los mensajes informativos, los avisos y los errores del programa para que el usuario pueda vigilar qué está haciendo el código.
Precondición	El sistema de mensajes se ha encendido correctamente al arrancar el programa principal.
Acciones	1. Una parte del código genera un aviso o un mensaje sobre la tarea que está realizando en ese instante. 2. El sistema le pega automáticamente la fecha, la hora, el nombre del archivo de código y el número de línea exacto. 3. El sistema mira si el aviso es lo bastante importante según lo configurado y lo pinta en la pantalla de la terminal.
Postcondición	El desarrollador ve en la pantalla de la terminal todo lo que hace el programa segundo a segundo.
Excepciones	Ninguna.
Importancia	Media

Tabla B.30: CU-6.1 Mostrar logs de ejecución.

CU-6.2	Almacenar historial de logs
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-6.2
Descripción	Escribe y guarda todos los mensajes que salen por pantalla en un archivo de texto físico en la carpeta logs del repositorio para poder revisarlos más tarde.
Precondición	La opción de guardar mensajes en un archivo está activada con el valor correcto en la configuración.
Acciones	1. El programa crea o abre un archivo de texto dentro de la carpeta de registros del proyecto. 2. Cada vez que sale un mensaje por pantalla, el programa escribe exactamente esa misma línea de texto dentro del archivo. 3. El programa guarda el texto usando un formato seguro compatible con letras especiales, tildes y caracteres complejos para evitar bloqueos.
Postcondición	Se guarda un archivo de texto físico con todo el historial de la ejecución.
Excepciones	Ninguna
Importancia	Baja

Tabla B.31: CU-6.2 Almacenar historial de logs.

CU-7	Visualizar datos
Versión	1.0
Autor	Nicolás Pérez Ibáñez
Requisitos asociados	RF-7, RF-7.1
Descripción	Permite al usuario abrir el panel web interactivo de Looker Studio para consultar de forma visual las gráficas con los datos extraídos de la provincia.
Precondición	El proyecto de Looker Studio está conectado a la base de datos que está actualizada con los últimos datos guardados.
Acciones	1. La plataforma de Looker Studio se conecta automáticamente con las tablas de la base de datos de BigQuery. 2. El panel carga y dibuja en la pantalla las gráficas de evolución, los mapas de ubicación y las listas de notas. 3. El usuario usa los botones y filtros de la pantalla para personalizar la visualización de la información a través de filtros.
Postcondición	El usuario analiza y explota los datos turísticos de la provincia.
Excepciones	Si internet o los servicios en la nube de Google sufren una caída, las gráficas se quedarán en blanco y mostrarán un aviso de error de conexión.
Importancia	Alta

Tabla B.32: CU-7 Visualizar datos.

Apéndice C

Especificación de diseño

C.1. Introducción

Este anexo describe cómo se ha estructurado técnicamente el sistema para cumplir con los requisitos funcionales detallados en los apartados anteriores. Define el plano de diseño del proyecto a través de tres pilares fundamentales: cómo se organizan los datos en la nube (diseño de datos), cómo se comunican las distintas herramientas y plataformas que forman el sistema (diseño arquitectónico) y cuál es el orden lógico exacto que siguen los procesos del programa durante una ejecución (diseño procedimental).

C.2. Diseño de datos

Para el almacenamiento permanente de la información, se ha seleccionado Google BigQuery, un almacén de datos en la nube que permite gestionar grandes volúmenes de registros y realizar consultas optimizadas de forma rápida. El modelo sigue una estructura relacional compuesta por seis tablas principales interconectadas entre sí y dos vistas SQL optimizadas para la herramienta de visualización.

A continuación, se describen los elementos que componen el modelo de datos:

- **municipalities:** Guarda la lista oficial de localidades de la provincia de Burgos, registrando su identificador del INE, su nombre, el número de habitantes y el control de fecha de las últimas actualizaciones de POIs y reseñas.

- **municipality_postal_codes:** Relaciona cada municipio mediante su identificador único con sus códigos postales oficiales posibles que pueden incluir puntos de interés de este municipio, permitiendo filtrar y descartar los POIs extraídos que pertenezcan a otros municipios.
- **categories:** Almacena la taxonomía oficial de Google Maps organizada en una jerarquía interna de cuatro niveles más la propia categoría oficial de Google Maps e incluye el nivel de segmentación demográfica.
- **pois:** Es la tabla central de puntos de interés. Contiene los lugares localizados con sus datos comerciales, estado de apertura, accesibilidad, coordenadas, el desglose de volumen de reseñas por cada estrella (de 1 a 5), un campo de tipo geométrico para mapas y el índice TORI calculado.
- **reviews:** Almacena los comentarios de los usuarios, guardando tanto el texto original como el traducido, el idioma detectado, el género estimado del autor mediante su nombre y los cálculos de sentimiento.
- **scraper_control:** Funciona como una tabla de auditoría interna que apunta la fecha y hora exactas de la última extracción completada con éxito para cada combinación de municipio y categoría, permitiendo saber y filtrar cuáles de esas combinaciones necesitan ser procesadas y cuáles no al ejecutar la aplicación.

Para facilitar el consumo de estos datos desde el cuadro de mando de Looker Studio sin penalizar el rendimiento ni generar costes innecesarios por consultas repetitivas, se han diseñado dos vistas SQL específicas:

- **view_pois:** Unifica la información de los POIs realizando cruces lógicos para adjuntar la población del municipio y los cuatro niveles de su categoría correspondiente.
- **view_reviews:** Consolida el histórico de reseñas combinando cada una con los datos del POI al que pertenece y la jerarquía taxonómica de su categoría.

Relaciones del modelo de datos

Las conexiones y dependencias lógicas entre las tablas de la base de datos se estructuran siguiendo los símbolos de obligatoriedad y cardinalidad definidos en el diagrama:

- **municipalities y municipality_postal_codes (1 mandatory to Many mandatory):** En la realidad geográfica un código postal

puede compartirse, pero en el modelo físico se resuelve con la tabla intermedia `municipality_postal_codes`. Cada fila vincula un único municipio obligatorio con un código postal, y todo municipio de la tabla maestra debe tener al menos un código postal asignado.

- **municipalities y pois (1 mandatory to Many optional):** Cada municipio de la tabla principal puede albergar varios puntos de interés en la tabla `pois`, quedando cada sitio vinculado de forma obligatoria a un único municipio a través del campo `id_municipality`. Es una relación opcional en el extremo del POI porque un municipio de la lista podría no tener ningún sitio extraído.
- **categories y pois (1 mandatory to Many optional):** Una categoría de la taxonomía sirve para clasificar múltiples lugares comerciales o turísticos, pero cada registro de la tabla `pois` pertenece obligatoriamente a una sola categoría de búsqueda mediante su `id_category`.
- **pois y reviews (1 mandatory to Many optional):** Un punto de interés puede acumular varias reseñas o no tener ninguna en la tabla `reviews`. Cada reseña individual de la base de datos se vincula obligatoriamente a un único POI mediante su identificador de Google (`poi_id`).
- **scraper_control (Tabla independiente):** La tabla `scraper_control` se ha diseñado como un componente aislado del resto de relaciones numéricas. Guarda las combinaciones de municipios y categorías directamente como texto plano (`STRING`) en lugar de usar claves foráneas. Esto se hace así porque el scraper en Python trabaja construyendo cadenas de texto para las búsquedas en la API de Google Maps. Guardar los registros en texto evita tener que hacer cruces de tablas (*JOINS*) costosos dentro de los bucles del programa cada vez que se quiere comprobar una fecha, mejorando la velocidad de ejecución y adaptándose al diseño desnormalizado que utiliza BigQuery.

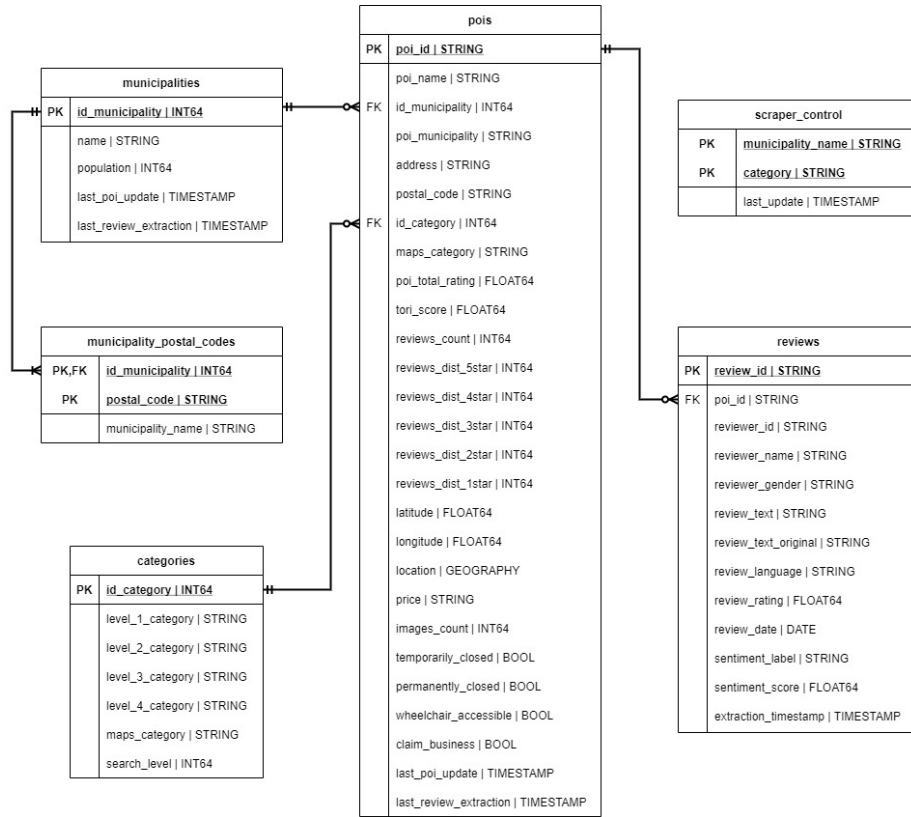


Figura C.1: Diagrama Entidad-Relación del modelo de datos en BigQuery.

C.3. Diseño arquitectónico

La arquitectura del sistema está organizada en cinco etapas o bloques principales que se conectan entre sí para formar el pipeline de datos, tal y como se detalla a continuación:

- **Desarrollo local:** Toda la programación del código se realiza en el entorno VS Code utilizando Python 3.13. Para evitar fallos de configuración y asegurar que el programa funcione igual en cualquier máquina, se incluye Docker para aislar el entorno de ejecución. Una vez que el código está listo, se usa el comando `git push` para subir los cambios al repositorio de GitHub.
- **Automatización y control (GitHub):** El repositorio del proyecto se aloja en GitHub. Desde aquí se utiliza GitHub Actions para programar y lanzar las ejecuciones automáticas del scraper a través de temporizadores o de forma manual. Además, se configuran las claves privadas

dentro de GitHub Secrets usando las variables `APIFY_TOKEN` y `GCP_SA_KEY` para proteger los accesos.

- **Extracción de datos (Apify):** Esta etapa se encarga de conectar el script de Python con la API de Apify para lanzar los actores encargados de extraer la información de los puntos de interés y las reseñas directamente desde Google Maps. Esta comunicación se realiza en dos pasos: primero, el pipeline lanza una **Consulta API (Python)** enviando los parámetros y filtros de búsqueda; segundo, el servidor de Apify procesa el rastreo en Google Maps y devuelve una respuesta con el conjunto de datos en un formato JSON con toda la información de los POIs y reseñas.
- **Almacenamiento (Google BigQuery):** Toda la información ya limpia y procesada se envía al almacén de datos de Google Cloud. Los datos se organizan en las seis tablas definitivas del sistema y se preparan las vistas SQL necesarias para la visualización.
- **Visualización (Looker Studio):** Es la capa final del proyecto, que consiste en un panel interactivo conectado directamente a la base de datos de BigQuery. Este cuadro de mando lanza consultas automáticas a las vistas de BigQuery cada vez que el usuario interactúa con la web, pintando los mapas y las gráficas estadísticas del Boletín de Turismo en tiempo real.

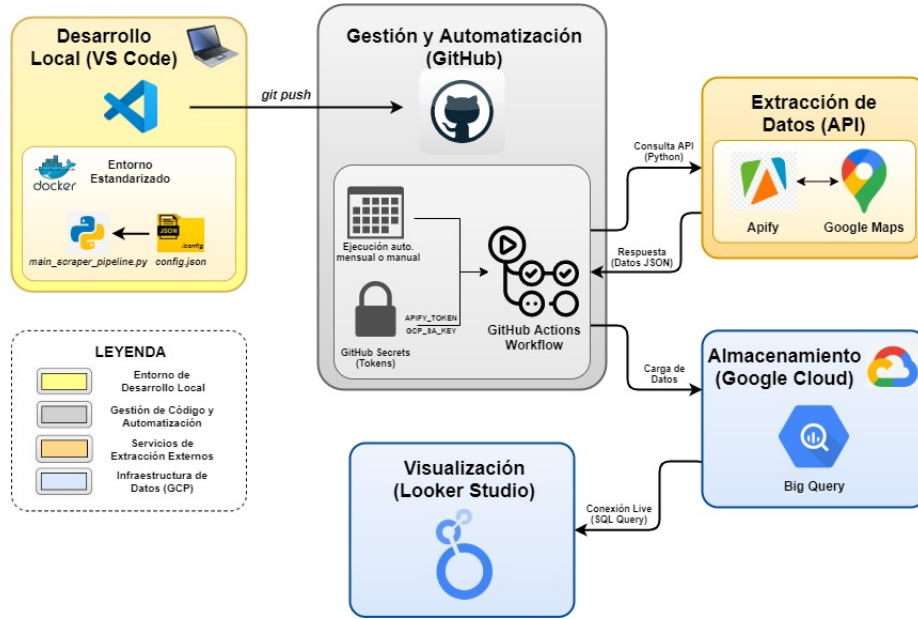


Figura C.2: Diagrama de arquitectura del proyecto.

C.4. Diseño procedimental

En este apartado se detalla el diseño procedimental del sistema, describiendo el flujo de ejecución cronológico y la interacción síncrona entre los módulos de software locales y los servicios en la nube (la API de Apify y Google BigQuery). Con el objetivo de garantizar la legibilidad de la arquitectura y debido a la densidad analítica del proceso incremental, el flujo se ha fragmentado en dos diagramas de secuencia complementarios: la Fase A, orientada al descubrimiento de POIs y control geográfico, y la Fase B, enfocada en la descarga masiva de reseñas, el procesamiento lingüístico de los datos y el cálculo del TORI.

El flujo completo del programa se organiza de manera secuencial siguiendo tres fases operativas principales distribuidas en dos bloques de secuencia lógicos:

- **Fase 1: Inicialización del sistema:** Al arrancar la aplicación, el script principal `main_scraper_pipeline.py` invoca al componente `config_loader.py` para leer y cargar en memoria el archivo `config.json` con los parámetros de la ejecución. Inmediatamente después, activa el subsistema de registros a través de `logger.py`. Para iniciar el flujo, llama al módulo `bigquery_loader.py` para realizar una consulta inicial

a Google BigQuery, descargando el mapa de categorías oficiales y la lista de municipios que tienen tareas de actualización pendientes según el margen de días configurado.

- **Fase 2: Descubrimiento de POIs (Fase A - POIs):** El pipeline inicia un bucle para recorrer uno a uno los municipios pendientes. En primer lugar, solicita a `bigquery_loader.py` los códigos postales autorizados de esa localidad para usarlos como filtro geográfico de seguridad. Si el término municipal requiere actualización, se calcula el nivel de búsqueda según la población y se recorren sus categorías invocando a la función `fetch_pois_from_apify` del módulo `apify_extractor.py` para descargar los POIs de Google Maps. El script principal procesa los resultados devueltos y envía la lista a `bigquery_loader.py`. Este módulo sube los datos a una tabla temporal de staging y ejecuta una sentencia SQL `MERGE` en el servidor para actualizar la tabla definitiva de POIs, registrando el control de la categoría y actualizando la fecha de control del municipio al haber sido ya procesados.

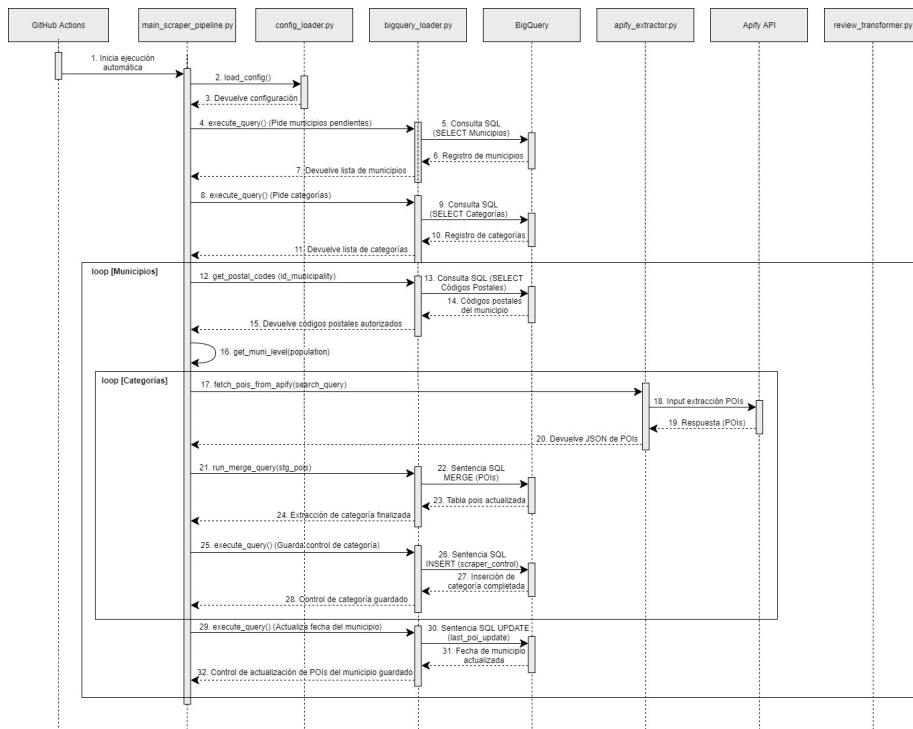


Figura C.3: Diagrama de secuencia: Inicialización y descubrimiento de puntos de interés (Fase A).

- Fase 3: Extracción y transformación analítica (Fase B - Reseñas):** Para cada punto de interés válido del municipio seleccionado, el archivo principal solicita a la base de datos la lista de POIs pendientes y, para cada uno, el identificador de la última reseña almacenada en ejecuciones previas para poder parar el procesamiento de reseñas cuando se encuentre una que ya está extraída (se extraen en orden de antigüedad). A continuación, llama al método `fetch_reviews_from_apify` para descargar las reseñas. Cada nueva reseña se envía al método `clean_translate_review` de `review_transformer.py` junto con su `poi_id` para descartar textos cortos, traducir reseñas de otros idiomas al castellano (o el idioma que se haya pedido en la configuración) de forma automatizada, estimar el género del autor por su nombre y calcular la nota de sentimiento. Finalmente, se invoca de nuevo al cargador para almacenar las reseñas con un `MERGE`, lanzar la consulta que recalcula el índice de reputación TORI del POI y guardar las marcas de control temporales tanto del POI como del municipio completo.

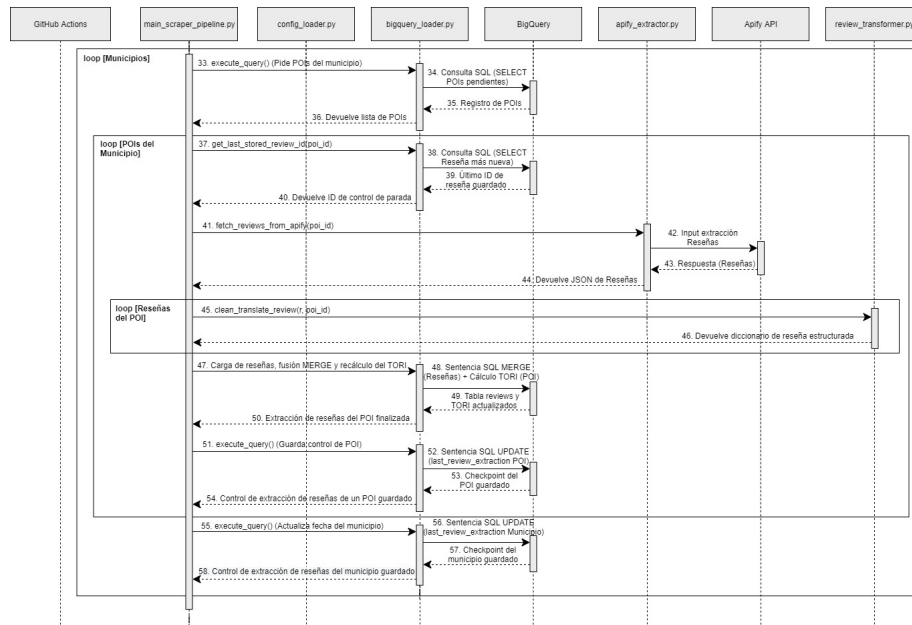


Figura C.4: Diagrama de secuencia: Extracción de reseñas, procesamiento NLP y persistencia (Fase B).

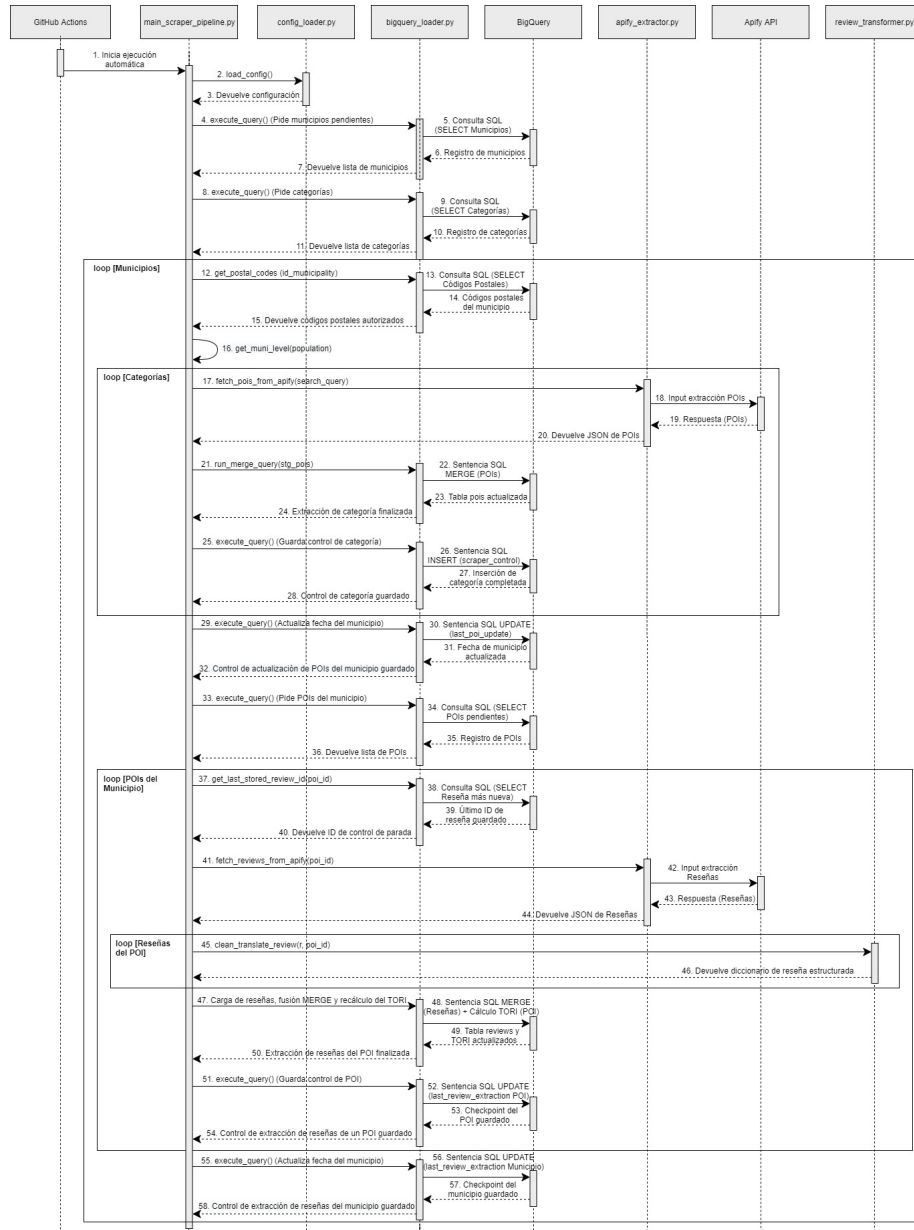


Figura C.5: Diagrama de secuencia completo.

Apéndice D

Documentación técnica de programación

D.1. Introducción

Este anexo contiene la información técnica necesaria para instalar, configurar y mantener el código del pipeline ETL. Está dirigido a desarrolladores que necesiten comprender la arquitectura del software o introducir modificaciones en el flujo de datos.

El proyecto está programado en Python y centraliza su lógica en la conexión con dos servicios externos: Google BigQuery, utilizado como almacén de datos persistente, y la API de Apify, empleada para realizar la extracción de los puntos de interés (POIs) y sus reseñas correspondientes.

D.2. Estructura de directorios y archivos

El repositorio está organizado dividiendo las distintas partes que forman el proyecto en carpetas fácilmente identificables. A continuación, se detalla el contenido de cada carpeta y el propósito de los archivos de la raíz.

Directorios del proyecto

- `.github/workflows/`: Contiene el archivo de configuración `run_scraper.yml` en formato YAML para automatizar tareas mediante GitHub Actions, permitiendo lanzar el scraper de forma programada o manual cuando se requiera dentro del propio repositorio de GitHub.

- **config/**: Almacena el archivo **config.json**, el cual centraliza toda la parametrización del pipeline sin necesidad de modificar el código fuente. Este fichero divide la configuración en bloques independientes que controlan las tablas de Google Cloud, las reglas de negocio (márgenes de días y umbrales de población), los filtros de ejecución manual y los parámetros de la API de Apify, la transformación de reseñas y los logs.
- **database/**: En esta carpeta se almacenan todos los elementos necesarios para reconstruir la base de datos utilizada para el correcto funcionamiento del sistema. Estos son: el archivo SQL (**schema.sql**) con las sentencias de creación de las tablas en Google BigQuery y los tres archivos CSV utilizados para cargar los datos iniciales de municipios, categorías y códigos postales de los municipios que se deben insertar una vez creadas las tablas.
- **docs/**: Almacena todos los archivos y recursos relacionados con la documentación técnica del TFG. Contiene la estructura completa del proyecto de Overleaf, incluyendo las imágenes (**img/**) y componentes individuales de la memoria y los anexos (**tex/**), así como sus propios ficheros fuente **.tex** y las bibliografías, así como los dos documentos finales ya generados en formato PDF.
- **logs/**: Carpeta destinada al almacenamiento de los logs generados en cada ejecución (si así se ha requerido) con el histórico de las ejecuciones, guardados cronológicamente para facilitar la búsqueda de errores o revisar el comportamiento del scraper.
- **src/**: Carpeta principal del código fuente en Python. Contiene los scripts encargados de ejecutar las fases del pipeline y una subcarpeta con utilidades comunes. La distribución exacta es la siguiente:
 - En la raíz de **src/** se encuentran los siguientes módulos controladores:
 - **main_scraper_pipeline.py**: Archivo principal que orquesta todo el flujo.
 - **apify_extractor.py**: Módulo encargado de la extracción de datos desde la API.
 - **review_transformer.py**: Script dedicado a la limpieza y transformación de las reseñas.
 - **bigquery_loader.py**: Script responsable de la carga de datos en la base de datos.

- `src/utils/`: Subcarpeta de componentes auxiliares compartidos por el resto de scripts, que contiene la inicialización del sistema de logs (`logger.py`) y la lectura del archivo de configuración JSON (`config_loader.py`).

Archivos de la raíz

Los archivos situados en la raíz del repositorio se encargan del despliegue, la seguridad y las dependencias del sistema. Su ubicación en este nivel responde a los estándares de desarrollo y configuración, ya que las herramientas de control de versiones, contenerización y automatización buscan estos ficheros de forma predeterminada en el primer nivel del directorio para coordinar el entorno global.

A continuación se detalla la función técnica de cada uno de ellos:

- `.dockerignore`: Indica qué archivos y carpetas locales no se deben incluir dentro de la imagen de Docker para evitar que el contenedor pese más de lo necesario.
- `.env`: Guarda las variables de entorno privadas. En este caso, el token de la API de Apify.
- `.gitignore`: Lista de archivos y carpetas que no se quieren subir al repositorio de forma predeterminada por seguridad.
- `Dockerfile`: Archivo con las instrucciones para construir la imagen de Docker del proyecto, definiendo el sistema operativo base y el entorno de Python.
- `docker-compose.yml`: Configura el contenedor mapeando los archivos de credenciales (`.env` y `key.json`) como volúmenes dentro del contenedor e inyecta la variable de entorno necesaria para la autenticación de Google Cloud.
- `key.json`: Archivo con las credenciales de la cuenta de servicio de Google Cloud necesarias para autenticar la conexión con BigQuery.
- `LICENSE`: Especifica los términos legales de uso y distribución del código.
- `README.md`: Presentación inicial del proyecto en la que se describe brevemente el objetivo, la arquitectura, los requerimientos del proyecto y los créditos del autor.

- **requirements.txt:** Lista de librerías de Python y sus versiones exactas requeridas para que el código funcione, agrupadas según su uso para la extracción, BigQuery, procesamiento de texto e infraestructura.

D.3. Manual del programador

Este apartado sirve como guía técnica para explicar el funcionamiento interno del código fuente del proyecto. Aquí se detalla el ecosistema de herramientas, la lógica algorítmica del archivo principal `main_scraper_pipeline.py`, la interacción entre los diferentes módulos de la carpeta `src/` y las estrategias de optimización aplicadas en tiempo de ejecución.

Entorno de desarrollo

Para poder trabajar con el código fuente, mantener el sistema o realizar modificaciones, el programador debe estar familiarizado con el siguiente conjunto de herramientas, lenguajes y plataformas que componen el proyecto:

Herramientas de software y lenguajes

- **Python (v3.13):** Es el lenguaje de programación central del proyecto. Se encarga de coordinar todo el pipeline ETL, procesar las llamadas de las APIs, limpiar los datos y ejecutar los modelos de Inteligencia Artificial.
- **SQL:** Lenguaje utilizado para interactuar con el almacén de datos de BigQuery. Se emplea dentro del código de Python para realizar las consultas de control, el filtrado de fechas y la fusión de registros mediante sentencias `MERGE`.
- **Visual Studio Code:** Entorno de desarrollo integrado (IDE) recomendado para trabajar en el proyecto debido a su compatibilidad con las extensiones de Python, Docker y entornos virtuales.
- **Git y GitHub:** Herramientas utilizadas para el control de versiones del código fuente, el almacenamiento del repositorio y la gestión del proyecto mediante su sistema de *Issues*.
- **L^AT_EX / Overleaf:** Herramientas empleadas para la redacción, maquetación y compilación técnica de la documentación y las guías del proyecto.

Infraestructura y automatización

- **Docker y Docker Compose:** Contenedor utilizado para empaquetar la aplicación con todas sus dependencias. Asegura que el scraper funcione exactamente igual en local que en el servidor, aislando el entorno de ejecución.
- **GitHub Actions:** Plataforma de integración y despliegue continuo encargada de automatizar la ejecución del scraper de manera periódica en la nube en el tiempo programado. Esto se realiza a través del archivo `.github/workflows/run_scraper.yml` que permite dos tipos de ejecuciones:
 - **Ejecución manual (`workflow_dispatch`):** Permite al programador entrar en la pestaña «Actions» del propio repositorio de GitHub y lanzar el pipeline de forma inmediata ejecutando el workflow, lo que es ideal para realizar pruebas de producción o actualizaciones de emergencia.
 - **Ejecución programada (`schedule - cron`):** Añadiendo el `schedule` con un cron, permite automatizar la ejecución del script en segundo plano utilizando el formato estándar de temporizadores de Unix (*Cron Job*), compuesto por cinco campos:
 - **Minuto (0 - 59):** Minuto de la hora en el que se lanza.
 - **Hora (0 - 23):** Hora del día en formato de 24 horas. GitHub Actions trabaja siempre con la hora UTC.
 - **Día del mes (1 - 31):** Día concreto del mes en el que se desea ejecutar.
 - **Mes (1 - 12 o nombres de tres letras como JAN-DEC):** Mes en el que se quiere ejecutar.
 - **Día de la semana (0 - 7 o nombres como SUN-SAT):** Donde el 0 y el 7) representan el domingo, el 1 el lunes, el 2 el martes, y así sucesivamente hasta el 6 (sábado).

Para todos los campos, se puede utilizar varios símbolos que permiten personalizar al detalle el tipo de automatización que queremos para el proyecto: El asterisco (*) significa «todos» (ej. todos los días de la semana); la coma (,) sirve para listar valores (ej. 1,3,5 para lunes, miércoles y viernes); el guion (-) define rangos (ej. 1-5 de lunes a viernes); y la barra (/) indica intervalos de repetición (ej. */2 para ejecutarse cada dos horas).

- **BigQuery:** Almacén de datos en la nube de Google Cloud Platform (GCP) donde se aloja la base de datos definitiva del proyecto. El dataset está compuesto por seis tablas:
 - **municipalities:** Contiene el listado oficial de los municipios de la provincia de Burgos, incluyendo sus datos demográficos (población) que el sistema utiliza para segmentar los entornos en rurales, semi-rurales o urbanos.
 - **categories:** Guarda la taxonomía o árbol jerárquico de categorías turísticas (clasificadas en varios niveles y su mapeo con las categorías nativas de Google Maps) que determina qué tipo de negocios debe buscar el scraper.
 - **pois:** Almacena los puntos de interés o establecimientos turísticos localizados, incluyendo toda la información relevante sobre cada uno de ellos que se extrae a través de Apify.
 - **reviews:** Contiene las reseñas con los datos más relevantes obtenidos y calculados sobre ellas.
 - **municipality_postal_codes:** Tabla de referencia que empareja cada municipio de la provincia con sus códigos postales válidos. Sirve como filtro de seguridad en la etapa de carga para descartar POIs que pertenezcan a otros municipios o provincias.
 - **scraper_control:** Es la tabla que gestiona el control incremental del sistema. Almacena las marcas temporales de la última vez que se actualizó con éxito cada combinación de municipio y categoría, permitiendo al pipeline saber qué trabajo ya está al día para poder saltárselo si se reinicia el programa.
- **Looker Studio:** Herramienta de visualización de datos conectada automáticamente a BigQuery, actualizándose de manera autónoma al producirse cambios en la base de datos.

APIs e integraciones externas

El pipeline depende de dos conexiones externas clave que requieren credenciales de autenticación privadas para funcionar:

- **Apify API:** Interfaz que permite invocar desde el código de Python el actor de Apify que realiza las búsquedas en Google Maps, delegando en esta plataforma la extracción masiva de POIs y reseñas.

- **Google Cloud Credentials (archivo `key.json`):** Archivo de claves en formato JSON que contiene los tokens de la cuenta de servicio de Google Cloud. Es imprescindible para que el código de Python pueda autenticarse de forma segura y realizar lecturas o escrituras en BigQuery.

Componentes de utilidad (`src/utils/`)

Estos componentes no ejecutan ninguna fase del pipeline de forma directa, sino que actúan como herramientas de soporte compartidas que utilizan todos los demás scripts del sistema para centralizar la configuración y controlar los registros.

Lector de configuración: `config_loader.py`

Este script es el encargado de leer el archivo centralizado `config/config.json` y convertirlo en un diccionario global de Python llamado `settings`.

Utiliza un método de clase para comprobar si la configuración ya ha sido cargada previamente en memoria. Gracias a esto, el archivo físico del disco solo se lee una vez al arrancar el programa. Si otros módulos del pipeline vuelven a importar las `settings`, el script les devuelve directamente el diccionario que ya está en memoria, ahorrando operaciones de lectura.

Si el archivo de configuración no existe en la ruta indicada o contiene un error de sintaxis en su formato, el script captura las excepciones correspondientes y lanza un aviso crítico que detiene el sistema, evitando que el pipeline funcione a ciegas.

Gestor de logs: `logger.py`

Este componente configura el sistema de registros de Python para monitorizar en tiempo real todo lo que ocurre en el pipeline, sustituyendo el uso de comandos simples de pantalla por un sistema estructurado de logs.

En primer lugar, extrae las opciones del bloque `logging_params` del JSON de configuración para aplicar el nivel de detalle seleccionado (como `INFO` o `DEBUG`) y definir el formato de texto de los mensajes, que incluye la hora exacta, el archivo y la línea de código donde salta el aviso.

El logger permite generar una salida de mensajes simultáneamente en dos destinos:

- **Consola:** Muestra los mensajes en la terminal en tiempo real durante la ejecución.
- **Archivo físico:** Si está activado en la configuración, genera un archivo de texto en la carpeta `logs/`. Para que sea único y fácilmente reconocible, nombra el archivo con la fecha y hora exactas del arranque. La escritura en el archivo de texto físico se fuerza explícitamente con codificación `utf-8`, porque las reseñas turísticas descargadas suelen contener tildes, caracteres extranjeros y emoticonos que harían fallar el programa si se utilizara la codificación por defecto.

Antes de generar los canales de salida, comprueba que no se hayan creado ya para evitar que, al importar el logger en los diferentes scripts del proyecto, los mensajes se dupliquen o tripliquen en la pantalla de la terminal y en el archivo de texto.

Configuración del sistema (`config/config.json`)

El comportamiento entero del pipeline se gestiona desde un único archivo centralizado en formato JSON. Al aislar estas variables en un archivo JSON en la raíz, se garantiza la portabilidad del sistema y se permite modificar el comportamiento del scraper o las credenciales de BigQuery de forma inmediata sin necesidad de alterar una sola línea del código fuente en Python.

Es fundamental no dejar en blanco ningún campo operativo del archivo `config.json` (salvo los selectores de filtrado que admitan cadenas vacías que se indiquen expresamente en este manual) para asegurar el correcto funcionamiento del pipeline y evitar excepciones por valores nulos en tiempo de ejecución.

A continuación, se detallan todos los parámetros disponibles estructurados por sus bloques operativos:

- **google_cloud:** Centraliza las credenciales de conexión con la infraestructura de Google Cloud Platform (BigQuery).
 - **credentials_file:** Nombre del archivo de clave privada en formato JSON (`key.json`) que contiene el token de la cuenta de servicio (SA) de Google Cloud para poder conectarse a BigQuery.
 - **project_id** y **dataset_id:** Identificadores del proyecto y del conjunto de datos en BigQuery de Google Cloud Platform (ej. `tfg-boletin-turismo-burgos` y `ds_turismo_reviews`).

- **tables:** Diccionario que mapea los nombres físicos de las 6 tablas del sistema (**municipalities**, **categories**, **pois**, **reviews**, **scraper_control** y **municipality_postal_codes**). Si los nombres cambian en la base de datos, basta con actualizarlos aquí.
- **tori_logic:** Establece los topes numéricos para calibrar el algoritmo del Índice de Reputación Online Turística. Permite personalizar la escala y el peso de cada puntuación, partiendo desde un mínimo de 0, siendo el máximo TORI total la suma de ambos máximos.
 - **max_score_rating:** Puntuación máxima o factor de ponderación asignado a la valoración media de Google Maps. (Ej. Si se pone 5 significa que la escala será de 0 a 5 y si se pone 10 será de 0 a 10 el valor que aportará al TORI total).
 - **max_score_sentiment:** Puntuación máxima o factor de ponderación asignado al análisis de sentimiento por IA de los textos (Funciona de la misma manera).
 - *Ejemplo de configuración:* Si se quiere dar más peso al sentimiento frente a la valoración media de Maps y usar una escala sobre 100, se puede configurar el rating en 40 y el sentimiento en 60, ofreciendo un rango de TORI final de [0, 100].
- **scraper_logic:** Define las reglas de negocio esenciales del scraper incremental y los criterios de segmentación demográfica.
 - **margin_days_update_pois:** Número de días que determina la caducidad de un punto de interés (POI). Si el tiempo transcurrido desde su última extracción supera este margen, el pipeline vuelve a invocar a Apify para actualizar los sitios del municipio o categoría. Si está dentro del margen, el bucle lo salta para ahorrar costes de API.
 - **margin_days_update_reviews:** Número de días que determina la necesidad de ampliar las reseñas de un POI. Si los días transcurridos desde la última actualización del campo correspondiente (municipio o POI) superan este margen, el pipeline vuelve a invocar a Apify para extraer nuevas reseñas del POI. Si está dentro del margen, el bucle lo salta para ahorrar costes de API.
 - **postal_code_prefix:** Prefijo numérico de la provincia (ej. "09" para Burgos) utilizado como filtro de seguridad para descartar sitios de otras regiones que se cuelen por error en las búsquedas.

- **location_province:** Cadena de texto que indica la provincia y el país en la que se pretende ejecutar el scraper (ej. "Burgos, España") para evitar ambigüedades con municipios homónimos en otras regiones.
- **muni_threshold_rural** y **muni_threshold_semi_rural:** Límites numéricos de población (habitantes) utilizados para segmentar los municipios en tres entornos de forma dinámica:
 - **Rural:** Población $\leq$ **muni_threshold_rural**.
 - **Semi-rural:** **muni_threshold_rural** < Población $\leq$ **muni_threshold_semi_rural**.
 - **Urbano:** Población > **muni_threshold_semi_rural**.
- **execution_filters:** Filtros avanzados orientados al control del flujo incremental, optimización de costes de la API y depuración ágil en local. La combinación de los parámetros de este bloque altera de forma directa el flujo algorítmico del pipeline, dando lugar a los diferentes modos de ejecución del sistema que se detallan en su apartado correspondiente.
 - **target_municipality / target_category / target_poi:** Cadenas de texto que actúan como selectores restrictivos para acotar la ejecución. Por defecto se configuran como un *string* vacío (" ") para que el pipeline procese de forma masiva e incremental toda la provincia. Si se introduce un valor en alguna de las tres, este debe coincidir exactamente con el nombre almacenado en la base de datos para realizar el filtrado (ej. "Miranda de Ebro", "Estación de tren" o "Cervecería Morito").
 - **order_municipalities_by / order_categories_by / order_pois_by:** Cláusulas de ordenación nativas en formato SQL que determinan la prioridad de recorrido de los bucles de ejecución del pipeline. Se deben escribir nombres de las columnas que coincidan exactamente con uno de los campos de la tabla que se está ordenando (si es de **municipalities**, usar una de sus columnas: **id_municipality**, **name**, **population**...). Ejemplos: "population ASC" / "population DESC": Para priorizar la extracción por municipios según su tamaño demográfico. "name ASC" / "id_municipality ASC": Para un barrido alfabético o numérico secuencial estricto y previsible.
- **apify_params:** Variables de configuración técnica requeridas por la API de Apify para modular el comportamiento de los web scrapers en la nube.

- **poi_actor**: Nombre del actor de Apify encargado del rastreo de lugares (ej. "compass/crawler-google-places").
- **reviews_actor**: Nombre o ruta del actor encargado de scrapear las reseñas (ej. "compass/google-maps-reviews-scraper"). Ambos campos permiten modificar el nombre de los actores de forma inmediata ante un posible cambio de nombre en Apify.
- **max_retries**: Número máximo de reintentos automáticos permitidos ante un fallo técnico momentáneo o un código de estado HTTP erróneo de la API de Apify.
- **retry_wait_seconds**: Tiempo de parada o latencia (en segundos) que el script esperará entre reintento y reintento.
- **language** y **country_code**: Códigos internacionales estandarizados en formato ISO ("es") para forzar a la interfaz de Google Maps a devolver los textos adaptados al lenguaje de destino indicado. En el caso del código del país es para realizar la extracción en el país indicado.
- **reviews_start_date**: Tiempo desde el que se quiere extraer reseñas de los POIs almacenados en la base de datos. Se puede establecer en horas, días o meses. Ej. "1 month" extraerá todas las reseñas de cada POI que se hayan publicado en el último mes.
- **reviews_sort**: Criterio de ordenación de las reseñas en el origen de Google Maps.
 - "newest": Trae primero las reseñas más recientes (ideal para actualizaciones incrementales rápidas).
 - "most_relevant": Trae primero las reseñas con más interacciones o interacciones valoradas por los usuarios de Google.
 - "highest_rating" / "lowest_rating": Ordena por la polaridad de las estrellas.
- **reviews_origin**: Canal de procedencia de las reseñas. Se puede establecer que solo extraiga reseñas de Google "google" o que además de Google provengan de cualquier otra página como Tripadvisor, etc. "all" (fijado por defecto en "google" para asegurar reseñas verificadas).
- **transformer_params**: Reglas de limpieza, preprocesamiento y validación aplicadas por el pipeline de Python sobre las reseñas extraídas antes de enviarlas a BigQuery.

- **min_review_words**: Umbral numérico de calidad (longitud de cadena). Si una reseña contiene menos palabras de las especificadas (ej. 8), se descarta automáticamente. Esto filtra comentarios vacíos o poco informativos (como *"Bien"* o un emoticono) garantizando un dataset de calidad de cara al análisis de sentimiento.
 - **target_lang**: Código de idioma de destino (*"es"*). Todas las reseñas extraídas se traducirán a este idioma antes de almacenarlas en la base de datos. Igualmente, en la base de datos se guardan tanto la versión original como la traducida.
 - **default_user_name, default_user_gender, default_review_lang**: Ej. *"Anónimo"* para el nombre del usuario y *"Desconocido"* para el género del autor e idioma de la reseña. Palabras que el script inyectará en las columnas correspondientes de la tabla de reseñas en caso de que la API recolecte campos nulos, corruptos o perfiles de usuario privados.
- **logging_params**: Controla el comportamiento del sistema de monitorización visual por consola.
- **log_level**: Nivel mínimo de severidad técnica que el logger capturará, pintará en pantalla y guardará en el archivo *.log*.
 - **"DEBUG"**: Muestra trazas detalladas de variables y flujos internos.
 - **"INFO"**: Muestra el flujo informativo normal de ejecución del pipeline.
 - **"WARNING"**: Filtra todo excepto advertencias de rendimiento o anomalías no críticas.
 - **"ERROR"**: Muestra únicamente excepciones que interrumpen o cuelgan módulos concretos del pipeline.
 - **log_to_file**: Variable booleana (*true* o *false*) que activa o desactiva la clonación de las trazas de la consola en un archivo de texto físico.
 - **log_format**: Máscara estructural con marcadores de posición nativos de Python que define el diseño visual exacto de cada línea del log (el formato por defecto contiene marcas de tiempo, etiquetas de severidad, nombre del script de origen, número de línea exacto donde ocurrió el evento y el mensaje descriptivo).

Flujo general del sistema (Algoritmo ETL)

Una vez descrito el entorno de desarrollo, las utilidades de control y las amplias posibilidades de configuración y modos de ejecución que ofrece el sistema, es el momento de detallar el núcleo algorítmico del proyecto.

El sistema funciona como un pipeline secuencial coordinado por el script `main_scraper_pipeline.py`. El objetivo principal del código es automatizar las fases de extracción, transformación y carga (ETL: Extract, Transform, Load), controlando en todo momento cualquier posible fuga de recursos y optimizando los costes tanto de tiempo de ejecución como económicos. Para la integración del flujo se importan los tres módulos especializados desarrollados en el proyecto:

Módulo de extracción: `apify_extractor.py`

Este módulo centraliza la conexión con la API de Apify. Su objetivo es recibir las peticiones de búsqueda desde el main, darles el formato de entrada adecuado que exige Apify y descargar los resultados. El script utiliza el token del archivo `.env` para inicializar el cliente `ApifyClient` y lee los identificadores de los actores desde el bloque `apify_params` de la configuración.

Un actor es un microservicio de la nube de Apify especializado en realizar una tarea de computación o extracción de datos de una web concreta. El pipeline interactúa con dos actores oficiales de la plataforma: uno para el descubrimiento de puntos de interés y la obtención de toda la información posible sobre cada uno de ellos y otro especializado en la extracción profunda de reseñas de un POI dado.

El flujo de comunicación entre el script y Apify sigue el siguiente esquema:

1. El código empaqueta las credenciales y los filtros operativos en un objeto JSON estructurado (el `run_input`) que define los límites de la búsqueda.
2. Al invocar el método `call()`, el cliente levanta una instancia del actor en los servidores de Apify. Este ejecuta de forma autónoma la navegación y extracción web sobre Google Maps utilizando la infraestructura de la plataforma.
3. Una vez finalizado el proceso con éxito, el Actor deposita los resultados en un dataset propio de Apify. El script realiza una petición de lectura

incremental mediante un iterador para descargar los datos y devolverlos al main.

El módulo cuenta con dos funciones principales:

- **fetch_pois_from_apify**: Se encarga de extraer los puntos de interés con toda su información. Como regla técnica de optimización, la función construye el diccionario de entrada (**poi_input**) de dos formas distintas según el tamaño del municipio:
 - **Para municipios de nivel 3 (ciudades)**: Añade el parámetro **locationQuery**, que divide el municipio en pequeñas áreas para hacer una búsqueda exhaustiva de POIs en cada zona. Esto es necesario porque con el flujo normal del bucle simple de categorías no es suficiente, ya que tiene una limitación de 20 POIs por extracción, siendo insuficiente para determinadas categorías que en las grandes ciudades pueden superar esa cifra de POIs en todo el municipio.
 - **Para municipios de nivel 1 y 2 (rurales o semi-rurales)**: Se pasa únicamente la cadena de búsqueda (**searchStringsArray**), que contiene la categoría y el municipio, porque con eso es suficiente para encontrar los puntos de interés de este tipo de municipios realizando ese flujo predeterminado.
- **fetch_reviews_from_apify**: Se encarga de extraer las reseñas de un POI concreto. Construye un diccionario de entrada (**rev_input**) donde genera una URL de inicio dinámica utilizando el identificador del lugar (**poi_id**) que se le pasa al actor para que obtenga las reseñas. Configura parámetros clave de la API como la fecha de inicio de las reseñas que queremos capturar, el idioma y el orden de preferencia de las reseñas (**reviewsSort**).

En ambas funciones, el código controla el número de intentos mediante la variable **attempt** hasta alcanzar el límite máximo configurado (**MAX_RETRIES**). Si la llamada al actor de Apify falla por microcortes de internet o saturación de la API, el script captura la excepción, emite un aviso de advertencia en los logs y suspende la ejecución durante unos segundos fijados en la configuración mediante un **time.sleep()**. Si tras agotar todos los intentos el error persiste, la función lanza una excepción definitiva hacia el main principal indicando que el proceso técnico de ese bloque ha fallado.

Módulo de transformación: `review_transformer.py`

Este componente concentra toda la carga analítica del pipeline a nivel de datos. Su función no es conectarse a servicios en la nube, sino recibir de forma individual las reseñas que extrae Apify para limpiarlas, traducirlas y enriquecerlas mediante modelos de Inteligencia Artificial antes de guardarlas en BigQuery.

Al inicializar el módulo, el script arranca en memoria dos analizadores globales reutilizables usando la configuración: el motor de análisis de sentimiento (`sentiment_analyzer`) y el detector de género por nombre (`gender_detector`).

El comportamiento de su función principal se detalla a continuación:

- **Filtro de longitud mínima:** Cuenta el número de palabras del comentario. Si la reseña viene vacía o tiene menos palabras que el mínimo configurado (por ejemplo, menos de 8 palabras), el script la descarta devolviendo un `None`. Esto evita almacenar texto que no aporta valor analítico (como simples exclamaciones o emoticonos).
- **Limpieza de texto:** Elimina los saltos de línea (`\n` y `\r`) para dejar el texto en una sola frase limpia. Además, borra las etiquetas automáticas que inyecta Google Maps cuando el usuario lee una reseña traducida.
- **Detección de idioma y traducción:** Utiliza la librería `langdetect` para adivinar el idioma real del texto escrito. Si detecta que no está en el idioma solicitado en la configuración, invoca a `deep-translator` de Google para traducirlo, manteniendo en la base de datos tanto el texto traducido como el texto limpio original.
- **Detección de género:** Extrae el primer nombre del autor de la reseña y lo pasa por la librería `gender-guesser`. Si el resultado contiene la palabra *"male"* lo clasifica como *"Masculino"*, si contiene *"female"* como *"Femenino"*, y si es ambiguo o una empresa, le asigna un valor por defecto (*"Desconocido"*).
- **Análisis de sentimiento por IA:** Envía el texto traducido al modelo predictivo de la librería `pysentimiento`. Este clasifica la opinión dando valores de 0 a 1 a las siguientes etiquetas (POS, NEU o NEG), sumando 1 en total. El script con esos datos realiza un cálculo matemático restando la probabilidad de que sea negativo a la probabilidad de que sea positivo:

$$p_{\text{score}} = P(\text{POS}) - P(\text{NEG})$$

Esto genera una nota continua en un rango exacto de $[-1, 1]$ que mide con precisión matemática el grado de satisfacción del autor de la reseña.

Por último, todos estos datos calculados y limpios se devuelven al main en un diccionario estructurado con todos los datos de la reseña que se van a guardar en la tabla de reseñas. Las claves de este diccionario deben coincidir exactamente con el nombre y el tipo de dato de las columnas de la tabla de BigQuery.

Módulo de carga: **bigquery_loader.py**

Este componente centraliza todas las comunicaciones con la base de datos de BigQuery. Su función es recibir los datos ya limpios desde el main y guardarlos en la nube.

Al arrancar, el módulo busca el archivo con la clave de GCP `key.json`. Si no lo encuentra, el programa escribe un aviso crítico en los logs y se detiene por completo para evitar realizar llamadas en balde a Apify.

Sus funciones principales son las siguientes:

- **execute_query**: Envía cualquier consulta SQL directamente a BigQuery y devuelve los resultados.
- **get_postal_codes**: Descarga la lista de códigos postales permitidos para el municipio que se esté procesando. Guarda estos datos en un conjunto y lo devuelve al main para que el filtro geográfico de la Fase A compruebe si el POI pertenece o no al municipio.
- **get_last_stored_review_id**: Busca en la base de datos la reseña más reciente que tengamos guardada de ese local y devuelve su ID con el fin de parar el guardado de reseñas en la base de datos al encontrarla (ya que se extraen en orden temporal) en el caso de que se ajuste en la configuración un tiempo de extracción de reseñas muy conservador para evitar que se pierda alguna, superando el tiempo de la última ejecución del scraper y, por lo tanto, extrayendo reseñas que ya se tienen almacenadas.
- **run_merge_query**: Es la función encargada de guardar los datos en BigQuery. Utiliza un **MERGE** que compara la tabla temporal con la tabla definitiva. La decisión de utilizar un **MERGE** mediante tablas temporales en lugar de **INSERT** o **UPDATE** se debe a que en lugar de

ir comprobando los datos uno a uno desde Python (lo que obligaría a hacer cientos de consultas lentas y costosas a los servidores de Google preguntando si cada ID ya existe), el sistema utiliza una estrategia optimizada:

- El script junta todos los datos y crea una tabla temporal en BigQuery para subirlos todos de una vez.
 - Se lanza la sentencia **MERGE** para que BigQuery cruce ambas tablas en sus servidores. En una sola operación, actualiza los locales que ya existían e inserta los nuevos.
 - El script borra inmediatamente la tabla temporal al terminar la fusión para no dejar residuos en la nube.
- **update_poi_tori**: Ejecuta una consulta para calcular automáticamente el TORI (*Tourism Online Reputation Index* o Índice de Reputación Online Turística) del punto de interés combinando la nota de valoración de estrellas de Google Maps y el análisis de sentimiento de las reseñas.

Estas puntuaciones vienen dadas en escalas diferentes: las estrellas van de 1 a 5 (amplitud de 4 puntos) y el sentimiento va de -1 a 1 (amplitud de 2 puntos). Para que ambas estén en la misma escala, el script realiza un escalado automático basándose en los topes fijados en la configuración (**max_score_rating** y **max_score_sentiment**).

- Los dos números indicados en la configuración representan el total de puntos que se le quiere dar a cada parte, permitiendo darle más importancia a una o a la otra y siempre escalando ambas puntuaciones para que el mínimo sea 0 en ambas y sea más fácil de ver y de comparar.
 - Ejemplo: Si a la valoración de Maps se le pone 7 y a la otra 3 la escala del TORI sería [0,10], pero dándole un 70 % de importancia a la puntuación de Maps y solo un 30 % al cálculo de sentimiento. Además de eso, el total del TORI va a ser la suma de ambas, por lo que se puede hacer una escala, por ejemplo, sobre 100 si así se requiere, indicando en los campos de la configuración una combinación de números que sume 100 (Ej: 50 y 50), dando un total de TORI de [0,100].
- El programa calcula en tiempo de ejecución las constantes **krating** (dividiendo el tope de valoración de Maps deseado entre 4, por la amplitud original de 4 puntos [1,5]) y **ksentiment** (dividiendo el

tope de valoración de sentimiento deseado entre 2, por la amplitud original de 2 puntos $[-1,1]$).

- A la nota de valoración media del POI de Google Maps (`poi_total_rating`) se le resta 1 para llevarla a base cero (haciendo que pase de un rango de 1-5 a un rango de 0-4) y se multiplica por su constante que ampliaría la escala al valor deseado en la configuración.
- Paralelamente, se calcula la media de los sentimientos de todas las reseñas del POI y se le suma 1 para eliminar los valores negativos (haciendo que pase de un rango de -1 a 1 a un rango de 0-2) y se multiplica por su constante que, de la misma forma, ampliaría la escala al valor deseado en la configuración.

La fórmula matemática definitiva que ejecuta la consulta de BigQuery de forma directa en el servidor es la siguiente:

$$\text{tori_score} = (k_{\text{rating}} \times (\text{poi_total_rating} - 1)) + (k_{\text{sentiment}} \times (\text{AVG}(\text{sentiment_score}) + 1))$$

Gracias a este diseño, el programador puede cambiar la escala o el rango final del TORI en cualquier momento simplemente tocando los toques en el archivo de configuración, sin necesidad de modificar ni una sola línea de código de la consulta SQL.

main_scraper_pipeline.py: Estructura de datos inicial y parámetros de control

Al arrancar, el main lee el diccionario global `settings` importado desde `config_loader.py` y mapea las variables de configuración en bloques operativos:

- `s_logic`: Controla los umbrales de población, los días de margen para que se deban realizar o no las actualizaciones y las constantes geográficas.
- `g_tables`: Guarda los nombres exactos de las tablas de BigQuery.
- `filters`: Contiene los filtros de ejecución manual utilizados por el desarrollador para realizar pruebas controladas.
- `dataset_path`: Define la ruta absoluta del conjunto de datos en la nube combinando el ID del proyecto y el ID del dataset de BigQuery.

Después, el script define la función auxiliar `get_muni_level(population)`. Este método segmenta el nivel de búsqueda del municipio en tres categorías en función de sus habitantes para optimizar las consultas y controlar los costes de la API: nivel 1 (Rural), nivel 2 (Semi-urbano) y nivel 3 (Urbano). Esta división se puede configurar estableciendo el límite de habitantes para cada categoría en el archivo de configuración.

main_scraper_pipeline.py: Flujo principal **(run_scraper())**

El método principal `run_scraper()` ejecuta un ciclo cerrado estructurado en bloques de inicialización, descubrimiento de POIs y extracción de reseñas.

Inicialización y carga de taxonomía

El programa calcula la fecha actual del sistema y recupera de la configuración los días de margen de actualización (`margin_days_poi` y `margin_days_reviews`). A continuación, realiza la descarga inicial de datos:

1. **Consulta de municipios:** Si el filtro `target_municipality` está vacío, se lanza una consulta SQL a BigQuery que extrae todos los municipios pendientes (aquellos cuyo marcador de actualización en la base de datos sea nulo o haya superado el margen de días establecido en la configuración). Si el filtro contiene el nombre de un municipio, la consulta se limita exclusivamente a este para obtener los resultados únicamente del municipio requerido.
2. **Mapeo de taxonomía:** Se descarga la tabla de categorías para construir en memoria un diccionario asociativo llamado `taxonomy` (`{maps_category: id_category}`) con el objetivo de asociar cada categoría oficial de Google Maps a un estándar propio y estructurado por niveles turísticos en la base de datos.

Fase A: Descubrimiento de puntos de interés (POIs)

Una vez realizada la configuración inicial, el main inicia el bucle principal que recorre la lista de municipios pendientes uno a uno. Para cada localidad, el script realiza las siguientes comprobaciones y preparaciones antes de llamar a la API externa:

- Se realiza una petición a `loader.get_postal_codes()` pasando el ID del municipio para descargar su lista de códigos postales permitidos

asociados a dicho municipio, con el fin de descartar posteriormente los POIs extraídos cuyos códigos postales no se correspondan con ninguno de los permitidos.

- El script evalúa si el municipio necesita actualizar su catálogo de POIs. Solo se accede a la Fase A si no se ha establecido en la configuración un único punto de interés concreto (No se quieren buscar POIs si no las reseñas de ese POI) y si la última actualización del municipio correspondiente es nula o ha superado el margen de días sin actualizarse.
- El sistema realiza una consulta a la tabla `scraper_control` para extraer la lista de categorías que ya han sido buscadas en el plazo estimado para este municipio y, de nuevo, evitar costes innecesarios.
- Llama a la función auxiliar para obtener el nivel del municipio (`muni_level`) según su volumen de población. A continuación, descarga desde BigQuery las categorías turísticas cuyo nivel de búsqueda sea igual o inferior al del municipio. Si el desarrollador ha especificado una categoría concreta en `target_category`, la consulta SQL se restringe únicamente a esa categoría, realizando una sola búsqueda en el municipio.

Con la lista de categorías filtrada para el municipio en cuestión, se inicia el segundo bucle interno que irá realizando la búsqueda categoría a categoría, construyendo la `search_query` que se le va a pasar a Apify para que realice la búsqueda delegando la tarea en el método del extractor `fetch_pois_from_apify()` que construye el input correspondiente y devuelve el listado de POIs que ha obtenido Apify para esa búsqueda.

Una vez que el módulo extractor devuelve la lista, el orquestador inicia un tercer bucle interno para procesar cada elemento de forma individual. Durante esta iteración, se aplican varios filtros de depuración antes de dar por válido el punto de interés:

- **Filtro de marcador genérico:** El script limpia el título del establecimiento y lo compara con el nombre del municipio. Si coinciden exactamente (o incluyen el sufijo ", *Burgos*"), el elemento se descarta, evitando almacenar los marcadores geográficos genéricos que genera Google Maps (uno por municipio solo con el nombre del pueblo que no es un POI real).

- **Sin código postal:** En caso de que el campo del código postal venga vacío desde la API, el script busca dentro de la dirección de texto si aparece alguno de los códigos postales autorizados del municipio. Si lo encuentra, lo rescata y lo asigna al registro de forma automática.
- **Validación de localización estricta:** El programa comprueba si el código postal definitivo del establecimiento se encuentra en la lista de permitidos del municipio y, si no lo está, lo descarta. En caso de que el municipio no tuviera códigos postales registrados en la base de datos, se aplica un filtro genérico para asegurar que el código postal comience por el prefijo de la provincia.
- **Gestión de categorías:** Comprueba si la categoría devuelta por Google Maps ya existe en el diccionario de taxonomía. Si es una categoría nueva, inserta la nueva fila en la tabla de categorías, agregándola en el nivel de turismo que se esté buscando en ese momento (por probabilidad de similitud) y actualiza el diccionario en memoria para no repetir la operación.

Una vez que el lugar ha superado todos los filtros, el script se encarga de organizar su información. El objetivo en este punto es coger los datos del JSON que devuelve Apify (que vienen con listas y subcarpetas internas) y guardarlos en un diccionario plano. Las claves de este diccionario deben coincidir exactamente con el nombre y el tipo de dato de las columnas de la tabla de BigQuery.

Finalmente, para completar la Fase A el script realiza los siguientes pasos:

1. Todos los POIs válidos de la categoría actual se acumulan en una lista y se cargan en una tabla temporal en la nube llamada `stg_pois_[timestamp]`.
2. Se invoca al método `loader.run_merge_query()` que ejecuta un MERGE que cruza la tabla temporal con la definitiva utilizando el `poi_id`. Esto permite insertar los puntos nuevos y actualizar los datos variables de los ya existentes.
3. Tras realizar la fusión con éxito, se inserta un registro en `scraper_control` marcando esa categoría y municipio como completados hoy.
4. Al salir de los bucles (si la ejecución no estaba limitada a una sola categoría), se hace un `UPDATE` a la tabla de municipios para marcar

también como completada la búsqueda de POIs en este municipio en la fecha de hoy, dando por cerrada la Fase A para esa localidad.

Fase B: Extracción de reseñas

El objetivo de esta fase es capturar de forma individual las reseñas de los usuarios para cada punto de interés:

Si no se indica un POI objetivo en la configuración, ejecuta una consulta en BigQuery para listar todos los POIs del municipio que nunca hayan sido procesados o cuya última extracción de reseñas supere el margen de días establecido. Si el filtro contiene un nombre, la consulta de reseñas se limita exclusivamente a ese lugar. Si no se encuentran POIs actualizables, salta al siguiente municipio.

Para cada POI seleccionado, se obtiene, a través del loader, el identificador de la reseña más reciente guardada en la base de datos para ese POI y se realiza la extracción. Las opiniones se descargan según el criterio fijado en la configuración. Si este criterio está establecido en **"newest"** (orden cronológico de más nuevas a más antiguas), el programa optimiza el proceso comparando los identificadores una a una a medida que las recorre. En el momento en que el identificador procesado coincide con el ID guardado en la base de datos, el script sabe que el resto de reseñas ya están almacenadas de ejecuciones previas y ejecuta un **break** para detener la lectura de ese POI en ese instante, evitando descargar duplicados y ahorrando costes en la API. Si se utiliza cualquier otro criterio de ordenación, este control de parada se omite para garantizar que no se queden reseñas nuevas sin procesar, delegando la eliminación de duplicados a la etapa de carga.

Una vez extraídas las reseñas, se pasa una a una por el método `transformer.clean_translate_review()` para limpiar el texto, traducirlo si no está en castellano, inferir el género del usuario y aplicar los modelos de inteligencia artificial para el análisis de sentimiento.

Finalmente, para completar la Fase B, el script realiza las siguientes acciones:

1. Todas las reseñas procesadas de ese POI se acumulan en una lista y se cargan en una tabla temporal en la nube llamada `stg_reviews_[timestamp]`, al igual que se hacía con los POIs.
2. Se invoca al método `loader.run_merge_query()` que ejecuta un MERGE cruzando ambas tablas para insertar las nuevas reseñas en la tabla definitiva de reviews.

3. Después, se recalcula de forma automática el TORI del sitio con las nuevas aportaciones de valoraciones y sentimientos de las nuevas reseñas añadidas.
4. Se hace un `UPDATE` en la tabla de POIs para marcar que sus reseñas se han actualizado con la fecha de hoy.
5. Al terminar con todos los puntos de interés, se actualiza también la tabla de municipios para marcar la extracción de reseñas como completada en el día de hoy, cerrando así la Fase B de ese municipio. Si solo se han extraído las reseñas de un POI indicado, no actualiza este campo en la base de datos porque del resto de POIs del municipio no se han extraído sus reseñas, por lo que no se puede marcar el municipio como completado.

Para asegurar que todo el proceso funcione de forma automática y desatendida, todo este bucle interno está envuelto en un bloque `try-except`. Si un punto de interés concreto da un error de red o falla al cargar sus reseñas, el script captura el fallo en los logs y utiliza un `continue` para saltar al siguiente local de la lista, evitando que un problema puntual detenga la ejecución general del pipeline.

La gran ventaja de este diseño es que el *scraper* está totalmente protegido en ambas fases contra cualquier fallo. Al ir guardando la información en la base de datos, municipio a municipio y activar los checks de control de las tablas (ej: `last_poi_update`), el sistema no pierde nada. Si el programa se corta a la mitad por un fallo de internet, todo lo que se ha extraído hasta ese segundo ya está guardado a salvo en la nube de Google.

Esto permite que, si tenemos que reiniciar el programa, el script lea esos checks y salte automáticamente todo el trabajo que ya está hecho. Así se evita empezar desde cero, lo que nos haría perder muchísimo tiempo y dinero en la API de Apify. Gracias a este control y a la cantidad de filtros que tiene el código, el pipeline funciona de forma segura y está preparado para cualquier situación sin malgastar recursos.

D.4. Compilación, instalación y ejecución del proyecto

El despliegue y la puesta en marcha del pipeline están pensados para que el sistema funcione exactamente igual tanto en un ordenador local en fase de

desarrollo como de forma automática en GitHub. El proceso de instalación consiste en preparar los servicios en la nube, configurar las claves de acceso y empaquetar el entorno dentro de un contenedor de Docker para evitar conflictos de versiones.

Para poner en marcha el proyecto por primera vez, se deben seguir estos pasos en orden:

Paso 1: Configuración del entorno de desarrollo

Para poder gestionar el proyecto, modificar sus parámetros y guardar los cambios de forma segura, se deben preparar las herramientas de desarrollo en el ordenador local:

1. **Instalación de Git (Control de versiones):** Descargar e instalar **Git** desde su página web oficial <https://git-scm.com/>. Durante el asistente de instalación en Windows, se recomienda mantener las opciones por defecto, asegurando que esté activa la configuración para permitir la ejecución de comandos desde la consola del sistema (*Git from the command line and also from 3rd-party software*) para que las terminales reconozcan las instrucciones nativas de control de versiones.
2. **Instalación del editor (Visual Studio Code):** Se debe descargar e instalar el entorno de desarrollo *Visual Studio Code*. Una vez instalado, servirá como el centro de mando para modificar el archivo de configuración `config/config.json`, gestionar las credenciales y abrir las terminales de comandos. Iniciar sesión en VS Code con la cuenta de GitHub.
3. **Creación del repositorio:** Acceder a GitHub y crear un repositorio completamente nuevo y vacío (sin archivo README ni `.gitignore`).
4. **Descarga y subida del código:** Descargar el código fuente original del proyecto en formato comprimido pulsando el botón verde "Code" y seleccionando "Download ZIP". Una vez descomprimido en el ordenador, se abre la carpeta en VS Code mediante el menú **Archivo > Abrir carpeta**.
5. **Inicialización y conexión del repositorio propio:** Para desvincular el proyecto de la ruta original y conectar la carpeta local de VS Code con el nuevo repositorio vacío creado en la nube, se debe abrir la terminal y ejecutar la siguiente secuencia de comandos:

- `git init`: Inicializa un contenedor de Git limpio en la carpeta local.
- `git add .`: Prepara todos los archivos del proyecto para el primer volcado.
- `git commit -m "Initial commit"`: Registra este estado inicial de los archivos. Si aparece un error de autenticación de usuario, hay que ejecutar los siguientes comandos: `git config --global user.email zou@example.com` y `git config --global user.name "Your name"`, con los datos correspondientes.
- `git branch -M main`: Establece la rama principal del proyecto.
- `git remote add origin <URL_del_repositorio_nuevo>`: Enlaza la carpeta local con la dirección web del repositorio personal creado en GitHub (sustituyendo la etiqueta por la URL correspondiente).
- `git push -u origin main`: Realiza la subida definitiva de todo el código a la nube. Aquí hay que autorizar el gestor de credenciales de Git, para permitir

Paso 2: Configuración de la base de datos (BigQuery)

El programa necesita que la base de datos ya esté creada antes de ponerse a buscar información. Para crearla, hay que entrar en la web de Google Cloud Console, crear un proyecto y montar las tablas utilizando los archivos que vienen dentro de la carpeta `database/`.

Crear el proyecto en Google Cloud

Entrar en la página de Google Cloud Console <https://console.cloud.google.com/welcome/new> e iniciar sesión con la cuenta de Google que disponga de una cuenta de facturación activa. En la barra de arriba, hay que hacer clic en el botón de "Selecciona un proyecto" y posteriormente, el botón "Proyecto nuevo". Se escribe el nombre del proyecto (el id que asigna BigQuery a ese proyecto será por lo que habrá que sustituir mi id del proyecto en los create table de schema.sql), se le indica que es un proyecto "Sin organización" (para evitar las restricciones de la organización que tiene una política que impide generar claves) y se da a "Crear". Una vez creado, el proyecto debería estar seleccionado en la barra de arriba.

Crear la estructura de las tablas

Ahora que está seleccionado el proyecto, se busca la herramienta **BigQuery** en el menú de la izquierda o en el buscador de arriba y se selecciona para crear ahí todo el almacén de datos del proyecto:

1. **Crear el Dataset:** En el panel izquierdo de BigQuery (En la pestaña de Explorador clásico), hay que hacer clic en los tres puntos verticales que salen al lado del id del proyecto y se elige la opción "**Crear conjunto de datos**". En el formulario que se abre, se le da un nombre al dataset (ejemplo para que cuadre y no tener que cambiarlo luego en las sentencias al crear las tablas: `ds_turismo_reviews`), en Ubicación de los datos hay que seleccionar `eu-rope-southwest1` (Madrid) se crea al hacer clic en el botón azul de abajo. Importante recordar cambiar los nombres que vienen por defecto tanto del proyecto como del dataset en el `config.json` por los elegidos en este paso (el id en el caso del proyecto).
2. **Abrir el editor de código:** Una vez dentro del dataset, pulsando el botón con el símbolo "+" (**Consulta en SQL**) que hay en la barra de arriba, se abre una consulta en blanco.
3. **Pegar el diseño de las tablas:** Se abre el archivo `database/schema.sql` en VS Code, se copia la totalidad de su contenido (las sentencias **CREATE TABLE** de las 6 tablas del sistema) y se pega en el editor de consultas de la web de BigQuery.
4. **Modificar consulta:** Importante sustituir todas las referencias al identificador del proyecto (`tfg-boletin-turismo-burgos`) por el ID del proyecto creado. Este identificador aparece en todas las sentencias **CREATE TABLE**, **CREATE VIEW** y en las referencias a tablas y vistas utilizadas en las consultas **FROM** y **JOIN**. En el caso de que se haya puesto un nombre diferente al dataset, también habrá que sustituirlo en todas las referencias. El formato debe ser siempre: `'id_proyecto.nombre_dataset.tabla'`.
5. **Ejecutar:** Le damos al botón "**Ejecutar**". Abajo mostrará un mensaje verde confirmando que todo está correcto y, en el menú de la izquierda, aparecerán las seis tablas creadas vacías.

Subir los datos iniciales (archivos CSV)

Para que el buscador sepa qué municipios de Burgos tiene que rastrear, qué códigos postales mirar y cómo clasificar las categorías de Google Maps, hay que rellenar las tablas de configuración con los archivos CSV de la carpeta `database/`.

Para la tabla `categories`, en el menú izquierdo de BigQuery, hacemos clic sobre el nombre que le pusimos al dataset y, en la barra de herramientas de arriba, le damos al botón "Crear tabla" y rellenamos el formulario con estas opciones:

- **Crear tabla desde:** Seleccionamos la opción "Subir".
- **Seleccionar archivo:** Buscamos en el equipo el archivo que corresponda (por ejemplo, `database/categories.csv`).
- **Formato de archivo:** Seleccionamos "CSV".
- **Nombre de la tabla:** Escribimos el nombre exacto de la tabla donde va ese archivo `categories`).
- **Esquema:** Marcamos la casilla de "Detección automática".
- **Preferencias avanzadas:** En **Preferencia de escritura** seleccionamos "Agrega a la tabla" y en **Filas del encabezado que se omitirán** hay que escribir el número 1 (esto sirve para que el programa ignore la primera línea del archivo, que es donde están los títulos de las columnas).
- Por último, pulsamos "Crear tabla".

Para la tabla de municipios (`municipalities`) y la de códigos postales (`municipality_postal_codes`) se deben seguir exactamente los mismos pasos, excepto en el apartado de **Esquema**. Para estas tablas hay que marcar la casilla de "Editar como texto" (desmarcando "Detección automática") y pegar exactamente lo siguiente (quitando el guión "que separa last_review_extraction"):

- Para la tabla `municipalities`: `id_municipality:INTEGER,name:STRING,population:INTEGER,last_poi_update:TIMESTAMP,last_review_extraction:TIMESTAMP`

- Para la tabla `municipality_postal_codes`: `id_municipality:INTEGER`, `municipality_name:STRING`, `postal_code:STRING`.

Esto es estrictamente necesario porque BigQuery, al usar la detección automática, interpreta los códigos postales del archivo CSV como valores numéricos (*INTEGER*) y los `TIMESTAMP` de `municipalities` como *STRING*. Al intentar subir estos datos como números a una tabla cuyo esquema ya ha sido definido previamente en el `schema.sql` como una cadena de texto (*STRING*), la plataforma lanza un error de incompatibilidad de tipos y bloquea la creación de la tabla. Forzando el esquema manual se soluciona este conflicto y se asegura que se respeten los formatos.

Al terminar estas tres subidas, la base de datos estará lista, con toda la información guardada y preparada para recibir los datos del scraper.

Paso 3: Obtención de credenciales y configuración de secretos en GitHub

Para que el pipeline pueda interactuar con los servicios externos de forma automática, es necesario extraer las claves de acceso de los proveedores en la nube y configurarlas tanto en el entorno de desarrollo local como en el repositorio de GitHub.

Extracción de la clave de Google Cloud

Para permitir que el script de Python se conecte a BigQuery y tenga permisos para escribir y modificar las tablas, se debe generar una clave de cuenta de servicio:

1. Dentro de la consola web de Google Cloud, abrir el menú de navegación de la izquierda y acceder al apartado `IAM y administración > Cuentas de servicio`.
2. En la barra superior, hacer clic en el botón `"Crear cuenta de servicio"`.
3. Asignar un nombre a la cuenta y pulsar `"Crear y continuar"`.
4. En el apartado de asignar roles, para otorgar los permisos de escritura necesarios, hay que seleccionar en el desplegable el rol de `Editor de datos BigQuery` y agregar un nuevo rol que es el de `Usuario de trabajo de BigQuery` que aparece dentro de la pestaña general `BigQuery`. Además, es necesario agregar un nuevo rol y hacer clic en `"Listo"`.

5. En la lista de cuentas de servicio que aparece en pantalla, hacer clic sobre la cuenta recién creada y acceder a la pestaña superior denominada "Claves".
6. Pulsar en el desplegable "Agregar clave > Crear clave nueva", seleccionar el formato **JSON** y hacer clic en "Crear".
7. El navegador descargará automáticamente un archivo de texto con la clave privada. Se debe mover este archivo a la raíz del proyecto dentro de VS Code y renombrarse como `key.json`.

Obtención del token de Apify

Para comunicar el script con la API que ejecuta los scrapers en la nube, es necesario extraer el token personal de la plataforma:

1. Acceder a la consola de Apify iniciando sesión.
2. En el menú de la izquierda, acceder al menú de configuración "Settings".
3. En las pestañas superiores, seleccionar la opción "API & Integrations".
4. Copiar la cadena de texto que aparece en el campo "Personal API tokens".
5. En VS Code abrir el archivo `.env` y escribir la variable pegando el token justo después del signo igual (sin espacios), con el siguiente formato:
`APIFY_TOKEN=token_copiado`. Guardar los cambios.

IMPORTANTE: Los archivos `key.json` y `.env` contienen las credenciales de acceso privadas que pueden generar costes económicos en las APIs. **Bajo ningún concepto se deben subir estos archivos al repositorio público de GitHub.** El archivo `.gitignore` incluido en el proyecto ya viene configurado para ignorarlos y bloquear su subida accidental.

Configuración de secretos en GitHub (GitHub Secrets)

Dado que los archivos de credenciales locales están bloqueados por seguridad y no se subirán a internet, se deben inyectar las claves directamente en el almacenamiento cifrado de GitHub para que el flujo automatizado de *GitHub Actions* pueda utilizarlas en sus ejecuciones:

1. Entrar en el repositorio creado en la cuenta de GitHub y acceder a la pestaña superior de configuración "Settings".
2. En el menú lateral izquierdo, desplegar la sección "Secrets and variables" y hacer clic en la opción "Actions".
3. Pulsar el botón verde superior "New repository secret" para dar de alta la primera clave:
 - **Name:** Escribir exactamente el nombre de la variable: `APIFY_TOKEN`.
 - **Value:** Pegar la cadena de texto del token copiado de la plataforma de Apify.
 - Hacer clic en "Add secret".
4. Pulsar de nuevo el botón "New repository secret" para dar de alta la segunda clave:
 - **Name:** Escribir exactamente el nombre de la variable: `GCP_SA_KEY`.
 - **Value:** Ejecutar en la terminal de VS Code el siguiente comando: `[Convert]::ToBase64String([System.IO.File]::ReadAllBytes("key.json"))` (quitar el guión que parte Bytes para el salto de línea) y copiar todo el bloque de texto que aparece en la terminal y pegarlo entero en este recuadro.
 - Hacer clic de nuevo en "Add secret".

Una vez completado este paso, el repositorio estará completamente preparado con los accesos necesarios para ejecutar el scraper en segundo plano de manera autónoma.

Paso 4: Entornos y opciones de ejecución del sistema

Una vez que la base de datos está lista y las claves de seguridad están configuradas en sus respectivos lugares, se debe preparar el entorno para poder ejecutar el código. El pipeline ofrece una arquitectura versátil diseñada tanto para el desarrollo y depuración en un entorno local como para la automatización desatendida en la nube. Independientemente de la opción de lanzamiento elegida, el comportamiento del script (los municipios a rastrear, el límite de reseñas o los filtros temporales) se gestiona de forma centralizada modificando los parámetros del archivo `config/config.json`:

1. Despliegue en entorno local (Desarrollo y pruebas)

Esta opción requiere la descarga del código fuente en la máquina del usuario y es la adecuada para realizar modificaciones rápidas, tareas de mantenimiento o pruebas de rendimiento controladas. Se puede preparar el sistema mediante dos alternativas:

Opción A: Instalación directa en el sistema Esta opción es la adecuada si se desea realizar modificaciones en el código fuente y probarlas de forma inmediata, evitando los tiempos de espera de compilación del contenedor.

1. **Instalación de Python:** Descargar e instalar **Python 3.13** desde su página web oficial <https://www.python.org/downloads/>. Durante el proceso de instalación en Windows, es indispensable marcar la casilla de verificación "Add python.exe to PATH" para que el sistema operativo reconozca los comandos desde cualquier terminal.
2. **Abrir la consola:** Acceder al menú superior de VS Code y seleccionar la opción `Terminal > New Terminal`.
3. **Instalar las librerías del proyecto:** Ejecutar en la terminal el comando de instalación de dependencias en la raíz del proyecto:

```
python -m pip install -r requirements.txt
```

El gestor de paquetes leerá el archivo e instalará en el sistema todas las librerías necesarias con las versiones exactas con las que se ha diseñado el proyecto.
4. **Lanzamiento :** El script se inicia ejecutando directamente en la consola el comando:

```
python main_scraper_pipeline.py
```

, estando dentro de la carpeta `/src`.

Opción B: Despliegue aislado mediante Docker Esta es la opción recomendada para asegurar que el pipeline funcione de forma idéntica en cualquier máquina o servidor de producción, ya que empaqueta el código, el sistema operativo y las librerías en un contenedor totalmente aislado, evitando tener que instalar dependencias a mano en el ordenador.

1. **Requisitos de almacenamiento y arranque:** Se debe descargar e instalar la aplicación *Docker Desktop* desde su página oficial <https://www.docker.com/products/docker-desktop/>. Una vez instalada,

se necesita tener instalado en Windows el subsistema de Windows para Linux para tener la máquina virtual en Docker (Ejecutar el comando `wsl.exe --install` en la PowerShell). Posteriormente, se debe arrancar y mantener encendida la aplicación en el ordenador. Es importante destacar que el sistema operativo anfitrión debe disponer de al menos **15 GB de espacio libre en el disco duro** antes de construir el contenedor. Este espacio es necesario para dar cabida a la máquina virtual de Linux (WSL2), la imagen base de Python 3.13 y el procesamiento de las librerías de análisis de texto.

2. **Construir y lanzar el entorno (Docker Compose):** Con la aplicación de Docker abierta, se accede a la terminal de VS Code y se ejecuta el comando en la raíz del proyecto:
`docker-compose up --build`
Este comando automatiza todo el proceso de golpe: lee el archivo de configuración `docker-compose.yml`, descarga la versión de Python 3.13, instala las librerías del `requirements.txt`, inyecta el token del archivo `.env` y arranca el script principal automáticamente en unos minutos.
3. **Ejecuciones posteriores:** Una vez realizado el proceso anterior por primera vez, el entorno quedará guardado de forma permanente dentro de la interfaz de *Docker Desktop*. Para futuras ejecuciones, (siempre y cuando no se haya modificado el código fuente del programa), no será necesario volver a usar la terminal, bastará con abrir la aplicación visual de Docker y hacer clic en el botón de reproducción (*Start*) para repetir el proceso con la imagen ya compilada. En caso de realizar cualquier modificación en el proyecto, será necesario volver a lanzar el comando `docker-compose up --build` desde la terminal para actualizar el contenedor con los nuevos cambios.
4. **Monitorización de logs:** Para monitorizar la ejecución en tiempo real, el usuario puede hacer clic sobre el contenedor en la interfaz de *Docker Desktop*, lo cual abrirá una consola integrada de *Logs* donde se vuelca todo el histórico de la ejecución. Igualmente, gracias a la configuración del `docker-compose.yml`, el archivo `.log` se almacenará en la carpeta de `/logs` del repositorio.
5. **Personalización de la nomenclatura en el despliegue:** Por defecto, Docker agrupa los contenedores utilizando el nombre de la carpeta raíz del proyecto y asigna un identificador automático al script (ej. `scraper-1`). Si se desea personalizar esta nomenclatura para adaptarla a estándares corporativos, se debe editar el archivo `docker-compose.yml`

introduciendo la directiva global `name:` en la raíz del archivo para renombrar el grupo, y la propiedad `container_name:` dentro del servicio específico para fijar el nombre del contenedor individual.

2. Despliegue en la nube (GitHub Actions)

A diferencia de los entornos locales, esta vía de producción ejecuta el sistema completo de forma remota y desatendida en los servidores de GitHub, sin requerir descargas de infraestructura, almacenamiento o consumo de recursos en el ordenador del usuario. Ofrece dos modos de activación:

Opción A: Ejecución manual Permite realizar ejecuciones en cualquier momento de forma manual en la nube:

1. Acceder al repositorio propio de GitHub y hacer clic en la pestaña superior "Actions".
2. En el menú lateral izquierdo, seleccionar el workflow correspondiente al pipeline del scraper (*Run Burgos Scraper*).
3. Hacer clic en el botón desplegable "Run workflow" situado a la derecha, seleccionar la rama principal (*main*) y pulsar el botón verde de confirmación "Run workflow" para iniciar la ejecución en la máquina virtual remota de GitHub Actions.

Opción B: Ejecución programada automática en la nube Es el estado de operación definitivo del sistema para trabajar de forma autónoma a largo plazo. Gracias a la directiva `schedule` integrada en el archivo YAML de configuración del workflow de GitHub Actions, la plataforma evalúa una expresión Cron temporal de forma constante. Al cumplirse el intervalo temporal configurado (por ejemplo, cada mes o cada semana), los servidores de GitHub despiertan el contenedor automáticamente, ejecutan el rastreo y actualizan las tablas correspondientes sin intervención humana.

D.5. Pruebas del sistema

El proyecto no cuenta con tests unitarios automatizados, pero su diseño permite realizar una amplia variedad de pruebas reales modificando los parámetros del archivo de configuración para obtener y comprobar resultados de elementos específicos.

El correcto funcionamiento de todo el pipeline y la detección de posibles fallos se controlan de forma activa mediante el volcado de mensajes por pantalla (logs) y el manejo de excepciones en los bloques de código.

Subsistema de Logs

El archivo `src/utls/logger.py` canaliza todos los mensajes informativos y de error que genera el programa. Esta salida se duplica de forma automática:

- Se muestra en tiempo real por la terminal de comandos, lo cual es esencial para monitorizar el contenedor Docker tanto en local como dentro de la plataforma GitHub Actions.
- Se guarda en archivos físicos locales dentro de la carpeta `logs/`. La nomenclatura de estos archivos utiliza el formato `ejecucion_YYYY-MM-DD_HH-MM-SS.log` para que cada ejecución genere su propio registro independiente y ordenado cronológicamente.

Control de Excepciones y Puntos de Control

Todas las consultas realizadas a la base de datos de BigQuery y las llamadas a la API externa de Apify están envueltas en bloques de control de errores (*try-except*).

Si ocurre una caída en la red, una saturación de peticiones o un fallo de autenticación, el programa captura la excepción, escribe el error detallado en el archivo de log para que el desarrollador sepa qué ha fallado y actualiza la tabla de control en BigQuery con los datos que se hayan guardado de forma segura hasta ese instante. De esta forma, se asegura que, si el pipeline se interrumpe por cualquier motivo, en el siguiente arranque saltará de forma automática los municipios y categorías que ya se completaron con éxito, impidiendo la duplicidad de registros y evitando un gasto innecesario de créditos en la API.

Matriz de Trazabilidad de Pruebas

Para verificar el correcto funcionamiento del sistema sin necesidad de pruebas unitarias aisladas, se han agrupado los requisitos funcionales en cuatro grandes escenarios de prueba de ejecución. A continuación, se muestra la correspondencia entre los requisitos y las pruebas diseñadas:

Escenarios de prueba de ejecución

Código de Prueba	Requisitos Funcionales Validados
PE-01 (Filtros y Configuración)	RF-1.2, RF-1.3, RF-1.5, RF-2.2, RF-6.1
PE-02 (Rastreo Geográfico)	RF-1.4, RF-3.1, RF-3.2, RF-3.3, RF-3.4, RF-3.5
PE-03 (Procesamiento y NLP)	RF-1.1, RF-3.6, RF-4.1, RF-4.2, RF-4.3, RF-4.4, RF-5.1, RF-5.2
PE-04 (Carga Cloud y Control)	RF-2.1, RF-5.3, RF-5.4, RF-5.5, RF-5.6, RF-6.2, RF-7

Tabla D.1: Matriz de correspondencia entre pruebas y requisitos funcionales.

Caso de Prueba	PE-01: Verificación de filtros y configuración operativa
Objetivo	Comprobar que el programa lee bien los filtros manuales y cambia el orden y el nivel de los mensajes por pantalla.
Entrada	Archivo <code>config.json</code> con <code>target_municipality: "Burgos"</code> , <code>target_poi: "Museo de la Evolución Humana (MEH)"</code> y <code>log_level: INFO</code> .
Pasos	1. Modificar los campos mencionados en el archivo de configuración externa. 2. Lanzar el script de forma manual desde la terminal de comandos. 3. Revisar el flujo de mensajes que se imprime en la consola.
Resultado Esperado	El programa arranca mostrando solo mensajes importantes (INFO). Ignora el resto de pueblos y salta directo a extraer únicamente las reseñas del Museo de la Evolución Humana sin extraer puntos de interés.
Estado	APTO (Aprobado)

Tabla D.2: Tabla de prueba PE-01.

Caso de Prueba	PE-02: Verificación del rastreo geográfico y filtros de zona
Objetivo	Validar que el bucle de municipios funciona, aplica bien las estrategias por tamaño de población y descarta los sitios de fuera de Burgos.
Entrada	Archivo <code>config.json</code> con los límites demográficos (400 y 5000 habitantes) y prefijo postal "09".
Pasos	1. Dejar los filtros vacíos para activar la ejecución sobre el listado de municipios. 2. Arrancar el pipeline y vigilar la llamada de categorías en los pueblos pequeños frente a las ciudades. 3. Comprobar los sitios descartados en el log.
Resultado Esperado	El programa itera en bucle por las localidades de BigQuery. Al llegar a un pueblo de menos de 400 habitantes, reduce la zona de búsqueda de Apify. Si un local devuelto por la API tiene un código postal que no pertenece a ese municipio, lo elimina de la lista.
Estado	APTO (Aprobado)

Tabla D.3: Tabla de prueba PE-02.

Caso de Prueba	PE-03: Verificación de limpieza, modelos NLP y control de duplicados
Objetivo	Comprobar que el programa filtra textos cortos, traduce, calcula el género, analiza el sentimiento, calcula el TORI y frena si la reseña ya existe.
Entrada	Archivo <code>config.json</code> con <code>target_poi: Catedral de Santa María de Burgos</code> para extraer las reseñas de la catedral (internacionales).
Pasos	1. Modificar los campos mencionados en el archivo de configuración externa. 2. Lanzar el script de forma manual desde la terminal de comandos. 3. Revisar la asignación de notas de sentimiento y género del archivo final. 4. Lanzar una segunda ejecución consecutiva para comprobar el freno por fecha.
Resultado Esperado	Los comentarios de menos de 8 palabras desaparecen. Las reseñas en otros idiomas se guardan traducidas al castellano. Se rellenan correctamente las columnas de género y sentimiento entre -1 y 1. En la segunda ejecución, el bucle hace un break inmediato al detectar que la primera reseña extraída ya existe en BigQuery.
Estado	APTO (Aprobado)

Tabla D.4: Tabla de prueba PE-03.

Caso de Prueba	PE-04: Verificación de carga en la nube, resiliencia y visualización
Objetivo	Validar que los datos se fusionan sin duplicar en BigQuery, se borran los residuos temporales, se guarda la fecha de control y Looker muestra los cambios.
Entrada	Lanzamiento desatendido por tiempo en GitHub Actions con datos listos para subir.
Pasos	1. Dejar que el Cron de GitHub Actions inicie el contenedor Docker. 2. Comprobar el dataset de BigQuery tras la finalización del programa. 3. Abrir el enlace del panel web de Looker Studio.
Resultado Esperado	La consulta MERGE añade los registros nuevos a la nube sin duplicados. Al terminar, la tabla temporal de staging desaparece del servidor. Se escribe la fila de control con el checkpoint y un archivo físico log en la carpeta correspondiente. El cuadro de mando web actualiza sus gráficos automáticamente.
Estado	APTO (Aprobado)

Tabla D.5: Tabla de prueba PE-04.

Apéndice E

Documentación de usuario

E.1. Introducción

El objetivo de este anexo es proporcionar la documentación de usuario del cuadro de mando desarrollado en Looker Studio [16], con el fin de describir su funcionamiento y facilitar la interpretación de la información representada en las diferentes pantallas que lo componen.

El cuadro de mando constituye la interfaz de visualización del sistema desarrollado, permitiendo explorar de forma gráfica e interactiva la información turística almacenada en BigQuery. A través de diferentes gráficos, mapas, tablas e indicadores, el usuario puede analizar la distribución de los puntos de interés (POIs), las características de las reseñas publicadas por los visitantes y el índice de reputación online (TORI), entre muchos otros aspectos de interés para el análisis turístico de la provincia de Burgos.

E.2. Requisitos de usuarios

El cuadro de mando desarrollado en Looker Studio es una aplicación web, por lo que no requiere la instalación de ningún software específico para su utilización. Únicamente es necesario disponer de un dispositivo con acceso a Internet y un navegador web actualizado para acceder al informe e interactuar con las diferentes visualizaciones que lo componen.

E.3. Instalación

Al tratarse de una aplicación web, el cuadro de mando desarrollado en Looker Studio no requiere ningún proceso de instalación ni la descarga de software adicional. El acceso al informe se realiza directamente a través del enlace web proporcionado por el autor.

E.4. Manual de usuario

El cuadro de mando está organizado en diferentes pantallas temáticas, cada una de ellas orientada al análisis de un conjunto específico de indicadores relacionados con los puntos de interés, las reseñas de los usuarios y el índice de reputación online (TORI).

La interacción con el cuadro de mando se realiza de forma completamente visual. Los gráficos permiten consultar información detallada al situar el cursor sobre sus diferentes elementos y, en aquellos casos en los que es posible, seleccionar categorías, municipios u otros valores para filtrar el resto de visualizaciones de la misma pantalla. Además, el informe incorpora controles de filtrado que permiten restringir el análisis a uno o varios municipios, años o categorías actualizando automáticamente los gráficos afectados.

En los siguientes apartados se presenta una descripción individual de cada una de las pantallas del cuadro de mando, detallando los elementos que las integran, la información que representan y las posibilidades de análisis que proporcionan.

E.4.1 Dashboard Overview

La primera pantalla, llamada *Dashboard Overview*, constituye la vista principal del cuadro de mando y ofrece una visión global de la información turística más relevante almacenada en la base de datos. Su objetivo es proporcionar un resumen de los principales indicadores del sistema y facilitar una primera exploración de los POIs antes de realizar análisis más específicos en el resto de las pantallas.

En la parte superior se muestran cuatro indicadores resumen que permiten conocer el estado general de la base de datos. Estos indicadores representan el valor medio del índice de reputación online (TORI), el número total de puntos de interés registrados, el número de reseñas disponibles y la calificación media otorgada por los usuarios. Todos ellos se actualizan automáticamente

cuando se aplica cualquier filtro sobre el informe, mostrando únicamente la información correspondiente al subconjunto de datos seleccionado.

La parte central de la pantalla está ocupada por una tabla interactiva que reúne la información más representativa de cada punto de interés. Para cada registro se muestran el municipio al que pertenece, la categoría y sus diferentes niveles de clasificación, el nombre del POI, el volumen de reseñas y el valor del índice TORI.

Esta tabla constituye el principal punto de consulta del cuadro de mando, ya que permite realizar comparaciones directas entre municipios, categorías y puntos de interés. Por ejemplo, el usuario puede identificar qué establecimientos presentan una mayor reputación online dentro de una misma categoría, localizar aquellos con un elevado volumen de reseñas o detectar diferencias entre municipios en función de la clasificación de sus recursos turísticos.

La pantalla incorpora, además, cuatro filtros correspondientes al municipio y los distintos niveles de categorías. Estos controles permiten restringir el análisis a un subconjunto concreto de la información, actualizando de forma automática tanto los indicadores resumen como la tabla. Gracias a ello, el usuario puede realizar análisis específicos sobre un municipio determinado o comparar el comportamiento de diferentes categorías de puntos de interés.



Figura E.1: Pantalla de vista general (Dashboard Overview).

E.4.2 Municipalities & Categories

La segunda pantalla, denominada *Municipalities & Categories*, está orientada al análisis de la distribución de los puntos de interés en función de los municipios y las categorías a las que pertenecen. Su finalidad es proporcionar una visión sobre cómo se reparte la oferta turística de la provincia y facilitar la comparación entre las diferentes zonas geográficas.

Esta pantalla, al igual que el resto, incorpora un filtro de municipios que permite restringir el análisis de la información a uno o varios municipios concretos, actualizando automáticamente todas las visualizaciones.

El gráfico *Category Distribution by Municipality* muestra la distribución de los puntos de interés entre las distintas categorías disponibles, permitiendo identificar cuáles tienen una mayor representación dentro del conjunto de datos o dentro de los municipios seleccionados.

Por otra parte, el gráfico *Nº of POIs by Municipality* representa el número de lugares registrados en cada municipio, facilitando la identificación de aquellos que concentran una mayor oferta turística.

Finalmente, el gráfico *Municipalities Contribution by Category* permite analizar cómo contribuye cada municipio al total de los POIs de cada categoría. Esta visualización resulta especialmente útil para comparar el peso relativo de los municipios dentro de cada categoría y detectar aquellas especializaciones o diferencias existentes entre ellos.

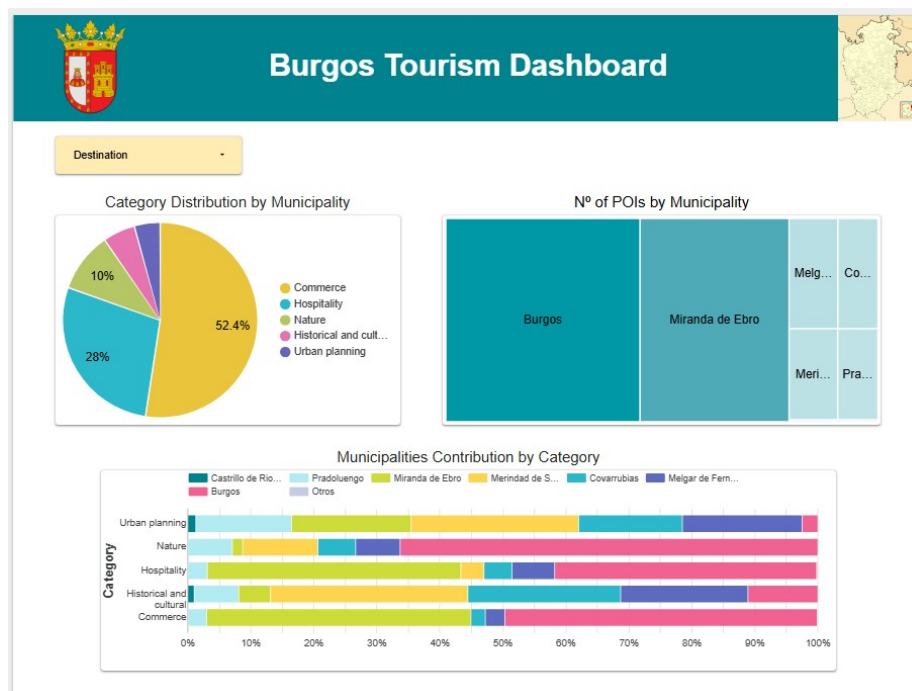


Figura E.2: Pantalla de análisis de Municipios y Categorías.

E.4.3 TORI I

La tercera pantalla, nombrada *TORI I*, está orientada al análisis del índice de reputación online (TORI) desde diferentes perspectivas, permitiendo comparar la valoración de los distintos POIs de la provincia. Además del filtro de municipios, la pantalla incluye dos indicadores resumen con el número de lugares analizados y el valor medio del índice TORI para la selección realizada.

Los gráficos *Average TORI Score by Category* y *Top 10 Municipalities by Average TORI Score* ofrecen una primera visión comparativa del índice TORI, mostrando qué categorías presentan una mejor reputación media y cuáles son los municipios con mayor valoración. De este modo, el usuario

puede identificar tanto los tipos de recursos turísticos mejor valorados como las localidades que destacan por la satisfacción reflejada en las reseñas.

Por otro lado, *TORI Score by Review Volume* permite analizar la relación entre el volumen de reseñas y el índice TORI de cada POI. Gracias a esta visualización es posible detectar lugares con una elevada reputación pese a contar con un número reducido de opiniones, así como establecimientos muy populares cuya valoración difiere del comportamiento general.

Finalmente, *TORI Score Distribution* complementa este análisis mostrando cómo se distribuyen los valores del índice TORI entre el conjunto de los POIs. Esto permite conocer cuáles son los rangos de puntuación más habituales y valorar si la reputación online se concentra en niveles altos, medios o bajos.

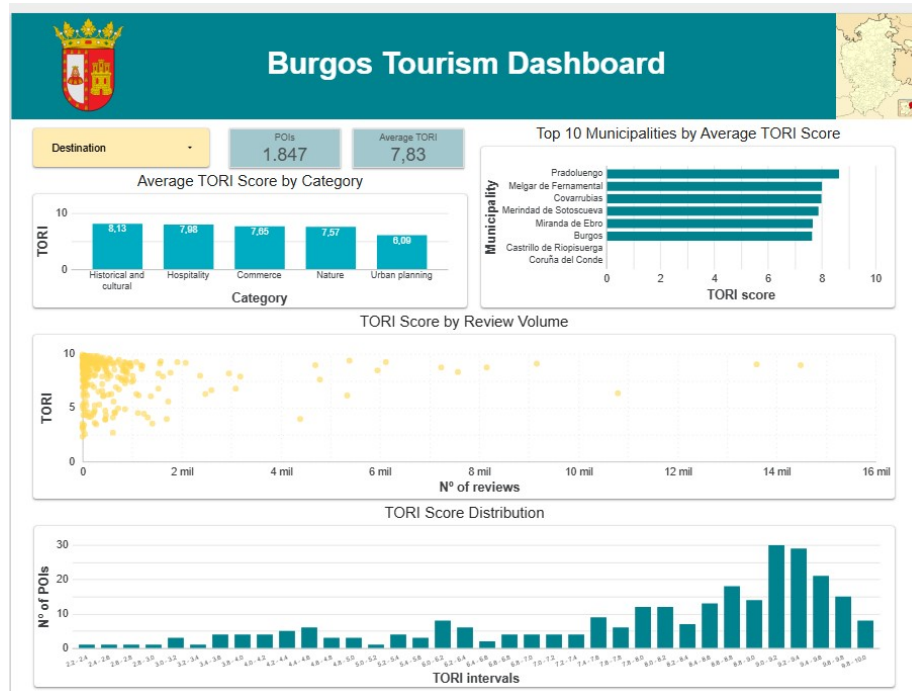


Figura E.3: Primera pantalla del análisis del índice TORI.

E.4.4 TORI II

La cuarta pantalla, denominada *TORI II*, complementa el análisis del índice de reputación online (TORI) estudiando su relación con diferentes variables asociadas tanto a los municipios como a los propios POIs. Además

del filtro de municipios, la pantalla incorpora dos indicadores resumen con el número de lugares analizados y el valor medio del índice TORI.

El gráfico *Average TORI Score vs N° of POIs* permite comparar la reputación media de los municipios en función del número de POIs registrados. Esta visualización facilita la identificación de municipios que, independientemente de su tamaño (el cual varía en función del número de población de cada municipio) o de la cantidad de recursos turísticos disponibles, destacan por presentar una reputación online especialmente elevada.

Por su parte, *POI Rating vs TORI Score* analiza la relación entre la valoración media otorgada por los usuarios y el índice TORI de cada POI. Gracias a esta comparación, el usuario puede comprobar si ambas métricas mantienen un comportamiento similar o identificar aquellos lugares cuya reputación online difiere de la valoración media recibida.

Finalmente, *Images vs TORI Score* muestra la relación existente entre el número de imágenes asociadas a cada POI y su índice TORI. Este gráfico permite explorar si los lugares con una mayor cantidad de contenido visual tienden a presentar una mejor reputación online o si ambas variables no muestran una relación significativa.

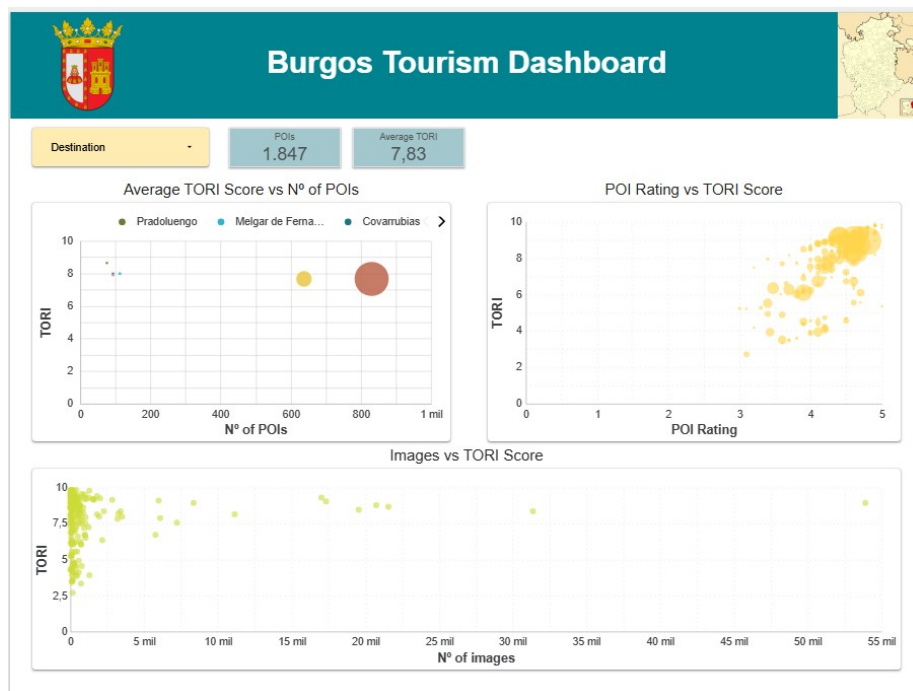


Figura E.4: Segunda pantalla del análisis del índice TORI.

E.4.5 Geographical Analysis

La quinta pantalla, llamada *Geographical Analysis*, permite analizar la distribución espacial de los POIs de la provincia, facilitando la identificación de las zonas con mayor concentración de recursos turísticos y de aquellos lugares que generan una mayor interacción por parte de los usuarios. Para ello, incorpora filtros de municipios y categorías que permiten centrar el análisis en un ámbito geográfico o temático concreto.

Los gráficos *POI Density Heat Map* y *Top 10 Municipalities by Number of POIs* ofrecen una visión complementaria de la distribución de los recursos turísticos. Mientras que el mapa de calor permite localizar de forma visual las áreas con mayor concentración de POIs, el gráfico de barras facilita la comparación entre los municipios con mayor número de lugares registrados, permitiendo identificar cuáles concentran la mayor parte de la oferta turística de la provincia.

Por otro lado, *POI Location* y *Top 10 Most Reviewed POIs* centran el análisis en los propios puntos de interés. El mapa de burbujas permite conocer la localización geográfica de cada POI, visualizar su nivel de actividad mediante el volumen de reseñas y obtener una primera aproximación a su reputación online. A su vez, esta información se complementa con el ranking de los lugares que acumulan un mayor número de reseñas, facilitando la identificación de los establecimientos con mayor visibilidad e interacción por parte de los usuarios.

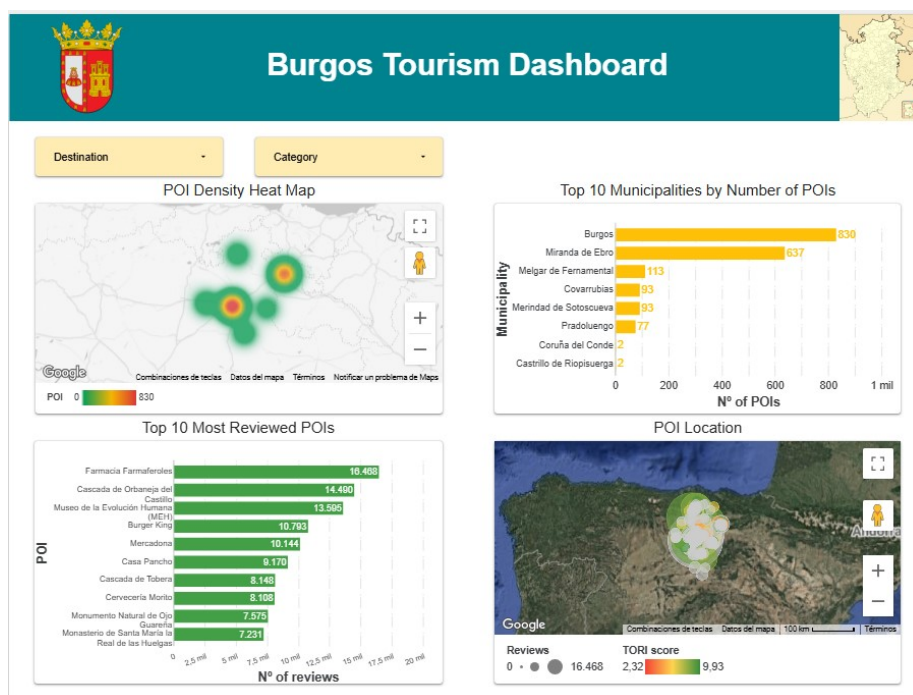


Figura E.5: Pantalla de análisis geográfico y espacial.

E.4.6 POIs Data

La sexta pantalla, que recibe el nombre de *POIs Data*, está dedicada al análisis de las características generales de los POIs registrados en la base de datos. Además del filtro de municipios, incorpora un indicador con el número total de lugares incluidos en la selección realizada.

El gráfico *POI Distribution by Level* permite explorar la distribución de los POIs según los distintos niveles de la clasificación utilizada en el proyecto. Mediante la navegación entre niveles a través de unas flechas que aparecen en la parte superior del gráfico, el usuario puede analizar la composición de la oferta turística desde una visión general (ver Figura E.6) hasta llegar a las categorías más específicas (ver Figura E.7), facilitando la identificación de los tipos de lugares con mayor representación.

Los gráficos *Wheelchair Accessibility* y *Claim Business* ofrecen información sobre determinadas características de los establecimientos. El primero permite conocer la proporción de POIs que disponen de acceso adaptado para personas con movilidad reducida, mientras que el segundo muestra el porcentaje de lugares cuya ficha de Google ha sido reclamada y gestionada

por su propietario, permitiendo obtener una visión general del grado de accesibilidad y de gestión de la presencia digital de los establecimientos.

Por último, los gráficos *Temporary Closure Status* y *Permanent Closure Status* permiten analizar el estado de apertura de los POIs. De forma conjunta, ambas visualizaciones facilitan la identificación del porcentaje de establecimientos que permanecen abiertos y de aquellos que se encuentran temporal o permanentemente cerrados, proporcionando una visión del estado actual de la oferta turística registrada.

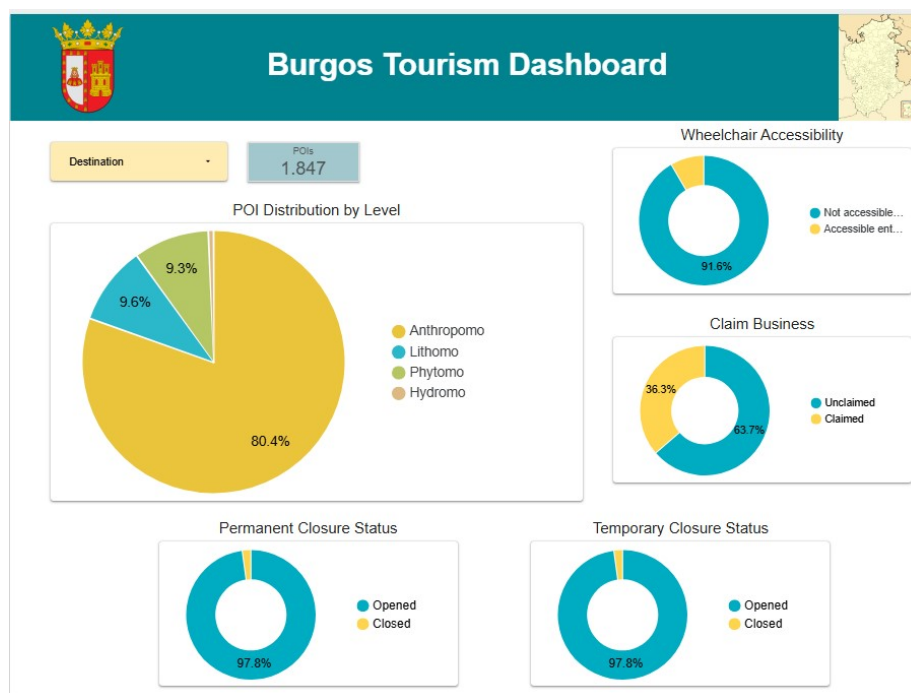


Figura E.6: Pantalla de datos generales y características de los POIs.

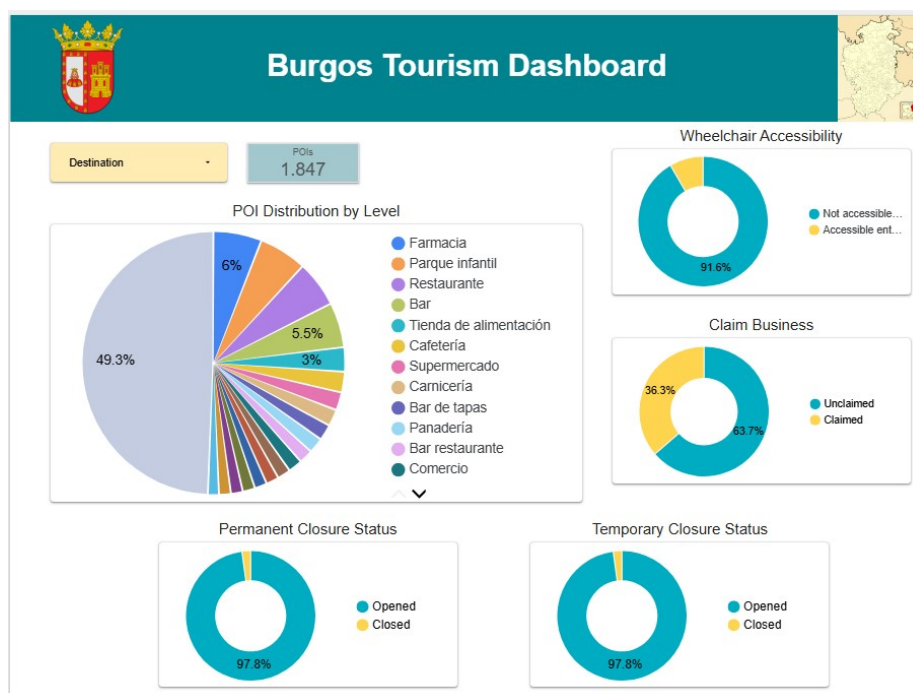


Figura E.7: Pantalla de datos generales y características de los POIs.

E.4.7 Review Analysis

La séptima pantalla, denominada *Review Analysis*, está orientada al estudio de las reseñas publicadas por los usuarios, permitiendo analizar tanto su volumen como la distribución de las valoraciones a lo largo del tiempo. Para ello, incorpora filtros de municipios y de años, abarcando este último un periodo comprendido entre 2022 y 2026, lo que resulta especialmente útil para centrar el análisis en un intervalo temporal concreto.

El gráfico *Reviews by Category* muestra el volumen de reseñas acumuladas por cada categoría de POIs, permitiendo identificar qué tipos de lugares generan una mayor interacción por parte de los usuarios y comparar el nivel de actividad entre las distintas categorías.

Los gráficos *Ratings Distribution by Category* y *Ratings Distribution* permiten analizar la distribución de las valoraciones otorgadas por los usuarios. Mientras el primero compara la proporción de valoraciones positivas, neutras y negativas entre las diferentes categorías, el segundo ofrece una visión global del conjunto de las reseñas, facilitando la evaluación del grado general de satisfacción de los visitantes y la detección de posibles diferencias entre categorías.

Por último, *Monthly Review Volume* representa la evolución del número de reseñas a lo largo del tiempo, permitiendo comparar la actividad registrada en cada mes y año. Esta visualización resulta especialmente útil para identificar patrones estacionales, analizar la evolución de la participación de los usuarios y detectar periodos de mayor o menor actividad.

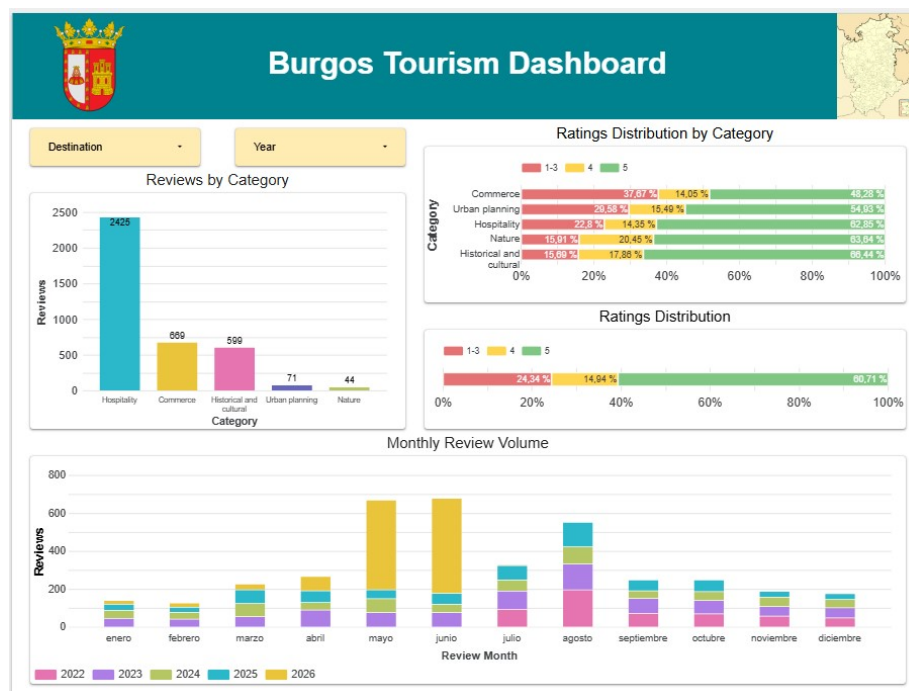


Figura E.8: Pantalla de análisis temporal e histórico de reseñas.

E.4.8 Review Demographics

La octava pantalla, nombrada *Review Demographics*, permite analizar las características de los usuarios que publican reseñas y estudiar sus patrones de participación. Para ello, incorpora los filtros de municipios y de años, facilitando el análisis de un periodo temporal o de una zona geográfica concreta.

Los gráficos *Gender* y *Average Rating Trend by Gender* permiten estudiar el comportamiento de las reseñas según el género de sus autores. Mientras que el primero muestra la distribución de las reseñas publicadas por hombres, mujeres y usuarios cuyo género no ha podido identificarse, el segundo analiza la evolución temporal de la valoración media otorgada por hombres

y mujeres, permitiendo comparar si ambos grupos presentan tendencias o comportamientos diferentes a lo largo del tiempo.

Por otro lado, los gráficos *Language*, *Reviews by Day* y *Review Language Flow by Weekday* ofrecen una visión conjunta de los hábitos de publicación de las reseñas. El primero permite conocer los idiomas predominantes en los que los usuarios escriben sus opiniones, el segundo muestra cómo se distribuyen las publicaciones a lo largo de la semana y el tercero relaciona ambas variables, facilitando el análisis de los idiomas más utilizados en cada día y la identificación de posibles patrones de comportamiento según el momento de publicación.

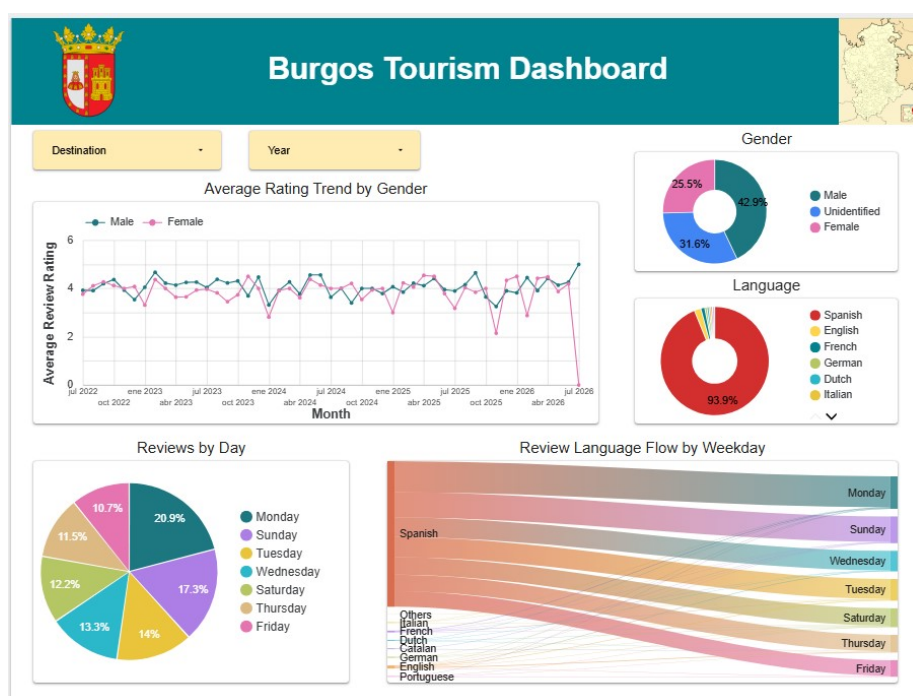


Figura E.9: Pantalla de datos demográficos de las reseñas.

E.4.9 Sentiment Analysis

Para finalizar, la novena pantalla, denominada *Sentiment Analysis*, está orientada al análisis del sentimiento expresado por los usuarios en las reseñas, permitiendo estudiar tanto la percepción general de los POIs como su evolución a lo largo del tiempo. La pantalla incorpora filtros de municipios y de años, además de dos indicadores resumen con el número de reseñas analizadas y la valoración media obtenida.

El gráfico *Sentiment Analysis by Category* permite comparar la cantidad de reseñas positivas, neutras y negativas registradas en cada una de las categorías de POIs. Gracias a esta visualización, el usuario puede identificar qué tipos de recursos turísticos concentran una mayor proporción de opiniones favorables o desfavorables y detectar posibles diferencias en la percepción de los turistas.

El gráfico *Sentiment Score Distribution* ofrece una visión global de la distribución del sentimiento obtenido tras el análisis de las reseñas, permitiendo conocer la proporción de opiniones con sentimiento positivo y negativo para el conjunto de la información analizada.

Por último, *Monthly Review Volume by Sentiment* y *Review Rating Evolution* permiten estudiar la evolución temporal de las opiniones publicadas por los usuarios. Mientras que el primero muestra cómo varía el volumen de reseñas positivas, neutras y negativas a lo largo del tiempo, el segundo representa la evolución de la valoración media de las reseñas, facilitando la identificación de tendencias y posibles cambios en la percepción de los usuarios durante los diferentes periodos analizados.

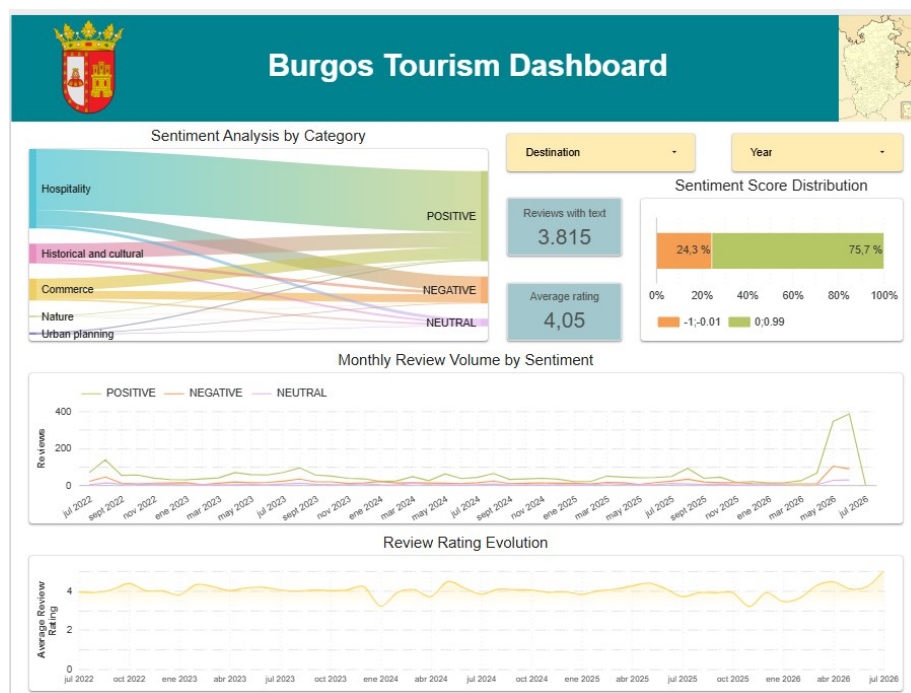


Figura E.10: Pantalla de análisis de sentimiento de las opiniones.

Anexo de sostenibilización curricular

F.1. Introducción

Hablar de sostenibilidad en el ámbito de la ingeniería informática no se limita únicamente a medir el consumo energético de los servidores o el hardware. Implica diseñar soluciones eficientes y útiles que aporten un valor real al entorno social y económico [4]. En este TFG, enfocado en automatizar y analizar los datos turísticos de la provincia de Burgos, la sostenibilidad se ha integrado desde el propio diseño del software. Optimizar el código para no desperdiciar recursos de computación y visibilizar zonas con menor impacto turístico permite alinear este proyecto con los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 [11]. De este modo, se demuestra cómo las herramientas de datos pueden ser un motor de cambio en el desarrollo local.

ODS 8: Trabajo decente y crecimiento económico - Impulso frente a la despoblación rural

La despoblación rural es uno de los problemas más graves que afronta la provincia de Burgos, afectando directamente al desarrollo económico de la región. El turismo sostenible es una vía clave para reactivar la economía de estos pequeños municipios. Al automatizar la recogida y el análisis de los puntos de interés (POIs) y sus reseñas en localidades con pocos habitantes, el sistema ofrece a las administraciones una información que, de forma manual, sería inviable o demasiado costosa de obtener. Conocer el comportamiento y

las opiniones de los visitantes en estas zonas permite a los gestores públicos diseñar campañas de promoción mucho más precisas y eficaces [13]. Esto ayuda a repartir los flujos turísticos fuera de las rutas habituales, impulsando los negocios locales, el comercio de proximidad y el empleo en municipios que sufren el declive demográfico.

ODS 9: Industria, innovación e infraestructura - Eficiencia tecnológica y relación coste-beneficio

Desde la perspectiva puramente técnica de la ingeniería de software, la sostenibilidad está ligada a la eficiencia de los algoritmos y al uso inteligente de la infraestructura cloud. El pipeline de datos se ha diseñado para funcionar estrictamente bajo demanda. Esto evita mantener servidores encendidos de forma continua y reduce el consumo energético innecesario durante los periodos en los que el scraper no necesita ejecutarse. La elección de una base de datos serverless como Google BigQuery optimiza el impacto ambiental al delegar la computación en una infraestructura global compartida y altamente eficiente. Además, el proyecto destaca por su alta viabilidad económica: con un coste de ejecución muy ajustado —gracias al aprovechamiento de los niveles gratuitos de GCP y al uso bajo demanda de Apify— se consigue procesar un volumen masivo de datos útiles para la gestión pública [14]. Innovar con arquitecturas eficientes demuestra que es posible desplegar soluciones estables a largo plazo sin sobrecargar los presupuestos institucionales.

ODS 11: Ciudades y comunidades sostenibles - Preservación y visibilización del patrimonio natural

El ODS 11 marca la importancia de proteger y poner en valor el patrimonio cultural y natural del mundo [12]. La provincia de Burgos posee una gran cantidad de espacios naturales, rutas y parajes que muchas veces carecen de la presencia digital necesaria para analizar su impacto real. Por ello, el pipeline incluye categorías específicas dedicadas al turismo rural y de naturaleza. Al recopilar y estructurar las opiniones de los usuarios sobre estos enclaves, el cuadro de mando en Looker Studio permite a los responsables de turismo evaluar cómo perciben los visitantes la conservación del entorno. Esta información ayuda a detectar problemas de masificación, planificar el mantenimiento de las infraestructuras o identificar qué áreas naturales despiertan más interés, promoviendo un modelo de turismo respetuoso con el medio ambiente y la biodiversidad local.

F.2. Conclusión y reflexión personal

El desarrollo de este TFG me ha servido para comprender que el desarrollo de software conlleva un impacto social directo que los ingenieros debemos asumir. Integrar estas competencias a lo largo del trabajo me ha enseñado que optimizar el código para reducir el tiempo de computación, usar estándares oficiales como el código de municipio del INE para evitar datos basura o plantear un almacenamiento en la nube eficiente, no son solo decisiones de ingeniería para ahorrar costes, son prácticas que minimizan la huella digital y aportan herramientas tecnológicas reales para combatir problemas tan complejos como la desigualdad territorial en nuestra provincia.

Bibliografía

- [1] Apify. Apify for universities and students. <https://apify.com/universities>, 2026. [Internet; 22-junio-2026].
- [2] Apify. Apify pricing plans. <https://apify.com/pricing>, 2026. [Internet; 22-junio-2026].
- [3] Apify Technologies. apify-client: The official apify api client for python. <https://pypi.org/project/apify-client/>, 2026. [Internet; registro de metadatos en PyPI; 23-junio-2026].
- [4] Crue Universidades Españolas. Directrices para la introducción de la sostenibilidad en el currículum. https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf, 2012. [Internet; directrices oficiales sectoriales; 23-junio-2026].
- [5] Í. Flores. Dataset de correspondencia de códigos postales y municipios del ine. <https://github.com/inigoflores/ds-codigos-postales-ine-es>, 2026. [Internet; 23-junio-2026].
- [6] GitHub. Acerca de la facturación para github actions. <https://docs.github.com/es/billing/managing-billing-for-github-actions/about-billing-for-github-actions>, 2026. [Internet; 22-junio-2026].
- [7] Google Cloud. Precios de bigquery. <https://cloud.google.com/bigquery/pricing>, 2026. [Internet; 22-junio-2026].
- [8] Google LLC. google-cloud-bigquery: Google bigquery api client library. <https://pypi.org/project/google-cloud-bigquery/>, 2026. [Internet; registro de metadatos en PyPI; 23-junio-2026].

- [9] Instituto Nacional de Estadística. Relación de municipios, códigos y denominaciones por provincias. <https://www.ine.es/daco/daco42/codmun/codmun08/08codmunmapa.htm>, 2026. [Internet; registro oficial de municipios; 23-junio-2026].
- [10] Jefatura del Estado. Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales. <https://www.boe.es/buscar/doc.php?id=BOE-A-2018-16673>, 2018. [Internet; Boletín Oficial del Estado; 23-junio-2026].
- [11] Naciones Unidas. La agenda 2030 para el desarrollo sostenible. <https://www.un.org/sustainabledevelopment/es/development-agenda/>, 2015. [Internet; marco global de objetivos; 23-junio-2026].
- [12] Naciones Unidas. Objetivo 11: Lograr que las ciudades y los asentamientos humanos sean inclusivos, seguros, resilientes y sostenibles. <https://www.un.org/sustainabledevelopment/es/cities/>, 2015. [Internet; metas del ODS 11; 23-junio-2026].
- [13] Naciones Unidas. Objetivo 8: Promover el crecimiento económico inclusivo y sostenible, el empleo y el trabajo decente para todos. <https://www.un.org/sustainabledevelopment/es/economic-growth/>, 2015. [Internet; metas del ODS 8; 23-junio-2026].
- [14] Naciones Unidas. Objetivo 9: Construir infraestructuras resilientes, promover la industrialización sostenible y fomentar la innovación. <https://www.un.org/sustainabledevelopment/es/infrastructure/>, 2015. [Internet; metas del ODS 9; 23-junio-2026].
- [15] Nidhal, B. deep-translator: A flexible free and unlimited python tool to translate between different languages. <https://pypi.org/project/deep-translator/>, 2026. [Internet; registro de metadatos en PyPI; 23-junio-2026].
- [16] Pérez I., Nicolás. Burgos tourism dashboard. <https://datastudio.google.com/reporting/57960b6b-29fb-41fa-84f3-e6e0fd5e770a>, 2026. [Internet; informe interactivo en Looker Studio; 7-julio-2026].
- [17] Pérez, J. M. y Juan Manuel, C. pysentimiento: Toolkit for sentiment analysis and social nlp tasks in spanish and english. <https://pypi.org/project/pysentimiento/>, 2026. [Internet; registro de metadatos en PyPI; 23-junio-2026].

- [18] Scikit-learn developers. scikit-learn: A set of python modules for machine learning and data mining. <https://pypi.org/project/scikit-learn/>, 2026. [Internet; registro de metadatos en PyPI; 23-junio-2026].
- [19] Unión Europea. Reglamento (ue) 2016/679 del parlamento europeo y del consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales. <https://www.boe.es/buscar/doc.php?id=DOUE-L-2016-80807>, 2016. [Internet; 23-junio-2026].
- [20] Unión Europea. Directiva (ue) 2019/790 del parlamento europeo y del consejo, de 17 de abril de 2019, sobre los derechos de autor y derechos afines en el mercado único digital. <https://www.boe.es/buscar/doc.php?id=DOUE-L-2019-80849>, 2019. [Internet; 23-junio-2026].